

Snapdragon® Multiverse Hackathon - Bangalore

Official Rules

PLEASE READ THE FOLLOWING OFFICIAL RULES CAREFULLY. BY CLICKING “ACCEPT”, YOU ACKNOWLEDGE THAT YOU HAVE READ, FULLY UNDERSTAND, AND UNCONDITIONALLY ACCEPT AND AGREE TO COMPLY WITH THESE OFFICIAL RULES AND ANY DECISIONS MADE BY SPONSOR, WHICH SHALL BE FINAL AND BINDING IN ALL RESPECTS.

Capitalized and/or italicized terms used but not defined in these Official Rules will have the meaning(s) ascribed thereto in Annexure A.

1. **Introduction**. These are the terms and conditions (“**Official Rules**”) for participation in the Snapdragon® Multiverse Hackathon (“**Hackathon**”) sponsored by Qualcomm Technologies, Inc., a Delaware corporation with a place of business at 5775 Morehouse Drive, San Diego, California 92121 (“**Sponsor**” or “**QTI**”).
2. **Skill-based Competition**. No purchase or payment of any kind is required to enter into or win the Hackathon. The Hackathon is not a prize draw, chit scheme or lottery and it is purely a competition of skill. The Hackathon is a contest of skill that involves computer application design and development based on Qualcomm chipset platforms.
3. **Dates and Times**. All dates and times hereunder are considered under India Standard Time.
 - a. **Phase 1 Submission Period**: May 27, 2026 (9:00 a.m.) – June 28, 2026 (11:59 p.m.)
 - b. **Phase 1 Judging Period**: June 29, 2026 (9:00 a.m.) – June 30, 2026 (12:00 p.m.)
 - c. **Shortlisting Announcement**: On or around July 1, 2026 (12:00 p.m.)
 - d. **Phase 2 Submission Period**: July 11, 2026 (1:00 p.m.) – July 12, 2026 (1:00 p.m.)
 - e. **Phase 2 Judging Period**: July 12, 2026 (1:00 p.m. – 5:00 p.m.)
 - f. **Winner Announcement**: On or around July 12, 2026 (5:00 p.m.)
4. **Eligibility**.
 - a. The Hackathon is open to:
 - i. individuals who are residents of the Republic of India, age eighteen (18) and above (“**Eligible Individuals**”); AND
 - ii. three (3) to five (5) Eligible Individuals participating as a team (“**Teams**”)(each Eligible Individual and Team are collectively, “**Entrant**” or “**you**”)
 - b. When a Team is entering the Hackathon, you must appoint and authorize one of the Eligible Individuals to represent, act, and enter the submissions, on the Team’s behalf (the “**Team Lead**”). By entering a submission on behalf of a Team, you represent and warrant that you are the Team Lead authorized to act on behalf of your Team.

- c. Each Entrant is responsible for securing any authorization or permission that may be required from his/her employer(s) in order to participate in the Hackathon. Sponsor may confirm that such authorizations have been duly obtained at any time during the Hackathon. In the event such authorizations have not been duly obtained, or a submission is found to be in violation of his/her/their employer's policies, Sponsor reserves the right to disqualify the Entrant or a Submission/Final Submission (defined hereafter) at Sponsor's sole discretion. Sponsor reserves the right to forfeit an Award (defined hereafter) if it becomes aware that the Entrant is not permitted to participate in the Hackathon and/or not authorized to grant the required license (defined in Section 8(c) below) by proper notification from his/her/their employer. Sponsor disclaims any and all liability for disputes arising between Entrants and their employers related to the Hackathon, which shall be the sole responsibility of Entrants.
- d. For clarity, it is Team's sole responsibility to ensure that its Team Lead and all other Eligible Individuals from the Team have reviewed, understood, and abided by his/her/their employer's policies regarding the eligibility to participate in the Hackathon. Entrants whose Team Leads or submissions found to be in violation of their employer's policies will be disqualified from the Hackathon.
- e. The Hackathon is NOT open to:
 - i. Individuals who are residents of any country other than the Republic of India.
 - ii. Individuals who are affiliated with any government agencies or state-owned entities.
 - iii. Individuals who are a *Restricted Person*.
 - iv. Organizations involved with the design, production, paid promotion, execution, or distribution of the Hackathon, including the Sponsor ("**Promotion Entities**").
 - v. Employees, representatives and *agents* of such Promotion Entities, and all *members of their immediate family or household*.
 - vi. Any other individual involved with the design, production, promotion, execution, or distribution of the Hackathon, and *each member of their immediate family or household*.
 - vii. Any Judge (defined hereafter), or company or individual that employs a Judge.
 - viii. Any parent company, subsidiary, or other *affiliate* of any organization described above.
 - ix. Any other individual or organization whose participation in the Hackathon would create, in the sole discretion of the Sponsor and/or Administrator, a real or apparent conflict of interest.

5. Hackathon - Phase 1.

- a. Registration. Entrants must register for the Hackathon by visiting <https://www.qualcomm.com/developer/events/snapdragon-multiverse-hackathon-bangalore> (“**Hackathon Site**”) and following all on-screen instructions to complete registration for Phase 1 of the Hackathon (“**Phase 1**”).
- b. Submission Requirements. By the end of Phase 1 Submission Period as set forth in Section 3, Team Lead shall complete all required fields and on-screen instructions to complete the proposal submission (each, a “**Submission**”). Each Submission must comply with the following “**Submission Requirements**”:
 - i. What to Submit: Entrants must submit a proposal that clearly aligns with the objectives of the Hackathon as outlined on the Hackathon Site.
 - ii. New & Existing: The proposal must be either newly created by the Entrant or, if the Entrant’s proposal existed prior to the Hackathon Submission Period, must have been significantly modified to add the AI models from the Qualcomm AI Hub or other open-source platforms.
 - iii. Limitations: The proposal must be the work and/or idea solely owned by the Entrant. Each Entrant must only submit one (1) Submission; if there are duplicate or multiple Submissions, only the first Submission would be considered eligible. Entrant cannot make any changes or alterations to your Submission after it has been properly submitted on the Hackathon Site. Please double check all fields in the intake form before submitting.
- c. Judges & Criteria. At the end of Phase 1, all eligible Submissions will be evaluated by a panel of judges selected by Sponsor (the “**Judges**”) during the Phase 1 Judging Period. Judges may be employees of Sponsor or third parties, may or may not be listed individually on the Hackathon Site, and may be subject to change before or during the Judging Period. Judging may take place in one or more rounds with one or more panels of Judges, at the discretion of the Sponsor, based on the following criteria (the “**Judging Criteria**”). Each criterion will be scored within a scale from one (1 = poor) to five (5 = excellent), with a total maximum of twenty-five (25) points:
 - i. Applicability to AI Use Case (1-5 points): Does the proposal clearly apply to multi-device AI scenarios (e.g., on device inference, real-time processing, web sockets)?
 - ii. Innovation and Creativity (1-5 points): Is the idea original or does it creatively apply existing multi-device AI technology in a new way? Does it show fresh thinking in how multi-device AI can be used?
 - iii. Feasibility of Execution (1-5 points): Is the proposal realistic to build within the twenty-four (24)-hour timeframe? Does the scope and complexity match what’s achievable in the event duration?
 - iv. Potential Impact (1-5 points): Could the proposed solution solve a meaningful problem or open up new possibilities in multi-device AI? Is there a clear benefit to users or society?

v. Clarity of Proposal (1-5 points)

6. Shortlisting Announcement.

- a. Assuming a sufficient number of eligible Submissions are received, the Judges will select up to fifty (50) eligible Submissions with the highest score as potential “**Shortlisted Entrants**” to advance to the onsite development phase (“**Phase 2**”).
- b. The decision of Judges and Sponsor with respect to the selection and confirmation of Shortlisted Entrants is final and binding in all respects and is not subject to challenge or appeal. If a Shortlisted Entrant is found to be ineligible, or if such Shortlisted Entrant has not complied with these Official Rules or declines to participate in any stage of the Hackathon for any reason, such Shortlisted Entrant will be disqualified, and an alternate Shortlisted Entrant with the next highest score may be selected. The final number of Shortlisted Entrant is subject to change at Sponsor’s discretion.
- c. If selected by the Judges as potential Shortlisted Entrants, each potential Shortlisted Entrant will be notified via e-mail by July 1, 2026, using the contact information provided at time of registration. Sponsor shall have no liability for any notification that is lost, intercepted or not received by the potential Shortlisted Entrant for any reason.
- d. At Sponsor’s sole discretion, each Eligible Individual of the potential Shortlisted Entrant may be required to execute and/or provide additional verification documents prior to advancing to Phase 2. If any potential Eligible Individual of the Shortlisted Entrant fails to sign and return such verification documents within five (5) business days, such potential Shortlisted Entrant may be disqualified as a whole at Sponsor’s discretion and an alternate Shortlisted Entrant may be selected.
- e. Sponsor may, at its discretion, hold an optional online session via Microsoft Teams (“**FAQ session**”). All confirmed Shortlisted Entrants are strongly encouraged to participate. During the FAQ session, Sponsor will provide material and/or information about Sponsor’s company, products/services, technology foresight and visions. Sponsor will also confirm the date and time, and the venue location for Phase 2 of the Hackathon.

7. Phase 2

- a. Orientation. All Shortlisted Entrants shall arrive at the Hackathon venue on time. Before Phase 2 starts, Sponsor will hold an orientation to introduce the platforms and tools that Sponsor expects Shortlisted Entrants to use to build their application prototype for Phase 2, such as Snapdragon® 8 Elite, Snapdragon® X Series Compute Platform, and Qualcomm® Cloud AI 100 (“**Platforms**”). Sponsor will also announce some housekeeping items such as the venue opening and closing times, the mechanisms, and the agenda for Phase 2. All Shortlisted Entrants must depart from the venue after the closing time per the venue rules or guidelines, but you may continue working on your application in your accommodation and may bring the Platforms with you to your accommodation, if needed. The next day, you must return to the venue by the announced time during the Orientation and bring back the Platforms to continue working on the application on site at the venue.

- b. Onsite Development. After the orientation, Shortlisted Entrants have twenty-four (24) hours to develop and build out an application prototype based on its Submission, leveraging the Platforms. The Phase 2 Submission Period can be extended per the Sponsor's discretion to accommodate technical difficulties or other unforeseen circumstances.
- i. Shortlisted Entrants shall submit their final designed application prototype by the end of Phase 2 Submission Period ("**Final Submission**"). No closed-source existing code is permitted; code must be open for consumption and use by all participants.
 - ii. If any modification or deviation are necessary from your submitted proposal while developing the application prototype, you must first notify Sponsor or its authorized representative and obtain its written consent before implementation. Any modification or deviation from your Submission without Sponsor's prior written consent may result in disqualification or forfeiture from this Hackathon.
 - iii. By advancing to Phase 2, Sponsor may at its sole discretion, and subject to your execution or acceptance of all applicable license agreements or other terms of use as determined by Sponsor, (i) provide you access to QTI Products (defined hereafter), which may be available at certain locations designated by Sponsor ("**Facilities**") or through remote access designated by Sponsor or its authorized representatives; (ii) provide certain assistance which may be provided in the form of access to hardware support, as described herein, to facilitate development of the proposal into a working prototype for the Final Submission. The term of such access to QTI Products may be continued through the last day of the Hackathon as published on the Hackathon Site, although Sponsor may designate a shorter or longer term of use for any particular Sponsor Products at its sole discretion.
 - iv. You acknowledge and agree that all use of QTI Products and access to Facilities shall be limited solely for the purpose of developing the working prototype for the Final Submission, unless otherwise specifically permitted by Sponsor. You further acknowledge that there may be other Shortlisted Entrants utilizing the Facilities and QTI Products during the course of the Hackathon. You shall ensure that your activities, and those of all persons under the direction or control of you, using the Facilities and QTI Products to work on the Final Submission and shall not cause any disruption to other Shortlisted Entrants in the Hackathon, especially with respect to the use of any shared or common locations and/or facilities such as meeting rooms, internet connectivity, computing, reprographic and telecom resources. Furthermore, you acknowledge that you shall be responsible for the payment of all taxes owed in relation to the receipt of any hardware or other benefits received pursuant to this agreement / Official Rules and any other agreement entered into by you with Sponsor or its affiliates in furtherance of the activities contemplated under this agreement / Official Rules, including without limitation, income, sales and use taxes, value-added tax,

- business tax, service tax, excise tax, consumption tax, customs duties or any similar charges or fees.
- v. To receive any QTI Products from Sponsor or its authorized representatives, you may be required to furnish and/or execute relevant documents and information required by Sponsor, including, but not limited to, (i) any requested tax documentation such as the necessary income tax withholding application, form, residency certificate, and tax code; (ii) any other information requested by Sponsor or its authorized representatives to permit Shortlisted Entrant to enter in the Hackathon or this agreement / Official Rules or to facilitate any of the transactions contemplated herein. Failure to furnish or execute required document or information in time may result in disqualification from the Hackathon or termination of this agreement/ Official Rules, at Sponsor's sole discretion.
 - vi. In addition to QTI Products that may be provided to you, you are also encouraged to bring extra devices powered by Snapdragon® technologies.
- c. Eligible Final Submissions shall meet the following criteria:
- i. The application shall not include any closed-source existing code; all codes shall be open for consumption and available to the public.
 - ii. The application must be provided in a GitHub repository, with the following files in addition to your code:
 - 1. A README file with the following information:
 - a. An application description;
 - b. Names and emails of all Eligible Individuals on the Team;
 - c. Setup instructions from scratch, including dependencies if applicable;
 - d. Run and usage instructions; and
 - 2. An open-source license (please refer to <https://choosealicense.com> for help choosing).
 - iii. The application must be runnable using your provided instructions.
 - iv. The application must be capable of being successfully installed and run on the Platforms for which it is intended and must function as depicted in the text description.
 - v. The application must be developed and/or commercially ready to the extent that it may be deployed on app store or other open source platform for users to download.
 - vi. The GitHub repository containing your application must be submitted by the end of the Phase 2 Submission Period. To submit your GitHub repository as your Final Submission, you must provide the link to your GitHub repository via a Microsoft Form submission. The Form to submit with a link to your GitHub repository will be provided by the Sponsor at the beginning of Phase 2.
 - vii. (Optional) – The following components are highly recommended but not mandatory:
 - 1. Tests and testing instructions to verify the app setup;

2. Notes section containing additional information not covered in the application description;
 3. References used while developing the application; and
 4. Well-commented code.
- d. Onsite Demonstration. By the end of Phase 2 Submission Period, all Shortlisted Entrants are required to demonstrate their application prototype in front of other Shortlisted Entrants and the Judges.
- e. Judges & Criteria. By the end of Phase 2 Submission Period, all eligible Final Submissions will be evaluated by Judges with the following criteria (“**Phase 2 Criteria**”), for a total maximum of one hundred (100) points. Judges may be employees of Sponsor or third parties, may or may not be listed individually on the Hackathon Site, and may be subject to change before or during the Phase 2 Judging Period. Judging may take place in one or more rounds with one or more panels of Judges, at the discretion of the sponsor.
 - i. **Technical Implementation (40 points)**
Evaluation based on resource utilization, optimization, latency and performance, and energy efficiency.
 - ii. **Application Use-Case and Innovation (25 points)**
Evaluation through the lens of problem solving, creativity and uniqueness, and user experience.
 - iii. **Deployment and Accessibility (20 points)**
Evaluation based on ease of installation and use.
 - iv. **Presentation and Documentation (15 points)**
Evaluation based on the clarity of explanation during the presentation, and code quality and documentation.
- f. Tie Breaking. If two or more Submissions are tied, the tied Submission with the highest score in the first applicable criterion listed above will be considered the higher scoring Submission. In the event any ties remain, this process will be repeated, as needed, by comparing the tied Submissions’ scores on the next applicable criterion. If two or more Submissions are tied on all applicable criteria, the panel of Judges will vote on the tied Submissions.
- g. Winner Announcement & Awards.
There will be a total of three (3) winners for this Hackathon. The winner for the Top Award (described below) will be the highest score voted by Judges based on Phase 2 Criteria, the winner of the Multi-Device Award (described below) will be the highest score voted by Judges based on Phase 2 Criteria AND a project that has the best use of multi-device orchestration, and the other winner of the Team’s Choice Award (described below) will be voted by all Shortlisted Entrants after listening to each other’s onsite demonstration of the respective Final Submissions. **Limitation: winners cannot be the**

same Entrant; if the winner of the Team’s Choice Award turns out to be the same Entrant as the one voted by Judges, then the Team’s Choice winner shall go to the second highest score voted by Judges based on Phase 2 Criteria. On or around 5 p.m. July 12, 2026, all winners will be announced on-site, with Awards being awarded on site, as follows:

- i. **Top Award (selected from Judges):** One (1) Copilot+ PC powered by Snapdragon X2 Elite for each member of the team for each member of the team (MSRP: \$1800 USD);
- ii. **Multi-Device Award (selected from Judges):** One (1) Meta Ray-Ban AI glasses for each member of the team (MSRP: \$329 USD);
- iii. **Popularization Award (voted by all Shortlisted Entrants):** One (1) Arduino UNO Q for each member of the team (MSRP: \$45 USD);

(collectively, “**Awards**”). AWARDS ARE AWARDED AS IS. The Manufacturer’s Suggested Retail Price (“**MSRP**”) stated herein are what the Awards will be sold as or are currently known to be sold for at the time the Official Rules were published and are subject to change. If an Award value is lower than the stated value at time of award, the difference will not be awarded. Awards do not include Wi-Fi data or service, which are the winners’ sole responsibility. Winners will be responsible for all sales and income taxes on the Awards. No substitution for Awards, except Sponsor reserves the right, in its sole discretion, to substitute Awards with a different Award of comparable value. Awards are not transferable or redeemable in cash, except at the sole discretion of the Sponsor. Winners are not recipient of Awards until Winners have been verified and/or notified by Sponsor on site. Sponsor does not make, nor is in any manner responsible for, any warranty, representation, expressed or implied, in fact or in law, relative to the quality, conditions or fitness of any aspects of Awards. Sponsor, at its sole discretion, may disqualify a potential winner if awarding an Award to the potential winner would create a real or apparent conflict of interest.

8. Intellectual Property.

- a. “**Intellectual Property Rights**”, as used herein, shall mean patents, rights to inventions, copyright and related rights, trademarks, trade names and domain names, rights in get-up, rights in goodwill or to sue for passing off, rights in designs, rights in computer software, database rights, rights in know-how and trade secrets and any other intellectual property rights, in each case whether registered or unregistered and including all applications (or rights to apply) for, and renewals or extensions of, such rights and all similar or equivalent rights or forms of protection which may now or in the future subsist in any part of the world.
- b. **Ownership.** All Intellectual Property Rights in the Submissions and Final Submissions, including any records, reports, documents, drawings, designs, photos, videos, portraits, graphics, logos, typographical arrangements, information, studies, object and source code, flow charts, diagrams, sketches, samples, prototypes, slides, rough work, clipart, proofs, manuscript, concept notes, ideas, strategy, algorithms, software, hardware, code, developments, outputs, results, technology or other materials arising out of or in connection with the Entrants’ participation in the Hackathon in whatever form

(collectively, “**Entrant Materials**”) shall vest solely with the Entrants, except for the Intellectual Property Rights that are originally owned by Sponsor, its affiliates, or third parties other than the Entrant.

- c. **License of Entrant Materials.** By registering for the Hackathon, each Entrant (including each Eligible Individual in a Team) hereby agrees to license all Entrant Materials under the MIT license located at <https://opensource.org/license/mit>. Entrants agree to execute any further documentation requested by Sponsor or its affiliates, contractors or agents in this regard (if any).
- d. **License of QTI Products.** Sponsor and/or its affiliates may from time to time and subject to execution of appropriate license agreements, provide Entrants a limited, restricted, non-exclusive, non-sublicensable, non-transferable, royalty free, revocable license to access or use certain software, hardware and/or infrastructure that is proprietary to Sponsor and/or its affiliates (collectively, “**QTI Products**”) for the duration of Entrants’ participation in the Hackathon solely for use in connection with development of the proposal for Submission and/or Final Submission (“**Purpose**”). Each Entrant acknowledges that access to and use of such QTI Products will be subject to applicable licensing terms and separate license agreements prescribed by the relevant licensing entity, and all use of QTI Products shall solely be for the Purpose. All rights, title and interests, including Intellectual Property Rights, in the QTI Products shall vest solely with Sponsor and/or its affiliates. Sponsor or the relevant licensors may at any time revoke the license without any further liability to the Entrant or any other party. For the avoidance of doubt, any use of QTI Products after termination or expiration of this agreement / Official Rules or the Hackathon, including as part of any outputs or results of the Submission or Final Submission, commercially or otherwise, shall be subject to separate agreements entered into with Sponsor or the relevant licensors or providers.
- e. **Exploitation of Ideas and Releases.** Each Entrant understands and acknowledges that:
 - (i) “**Qualcomm entities**”, including Sponsor and its affiliates, have wide access to ideas, concepts, prototypes, products, code, know-how, trade secrets, plans, designs and construction information, processes and flowcharts, formulas, manufacturing, techniques, discoveries, inventions, product specifications, machinery, drawings, photographs, computer source codes, equipment, devices, tools and apparatus and any other engineering or other technical information, and other materials, and the same are constantly being submitted to it or being developed by its employees/personnel; (ii) many ideas, concepts, strategies may be competitive with, similar to, or identical to content in the Submission or Final Submission, any prototypes or Entrant materials and/or each other in theme, idea, workflow, concept, design, format, or other respects; (iii) the Entrant will not be entitled to any compensation as a result of Qualcomm entities’, including Sponsor or its affiliates’ use of any such similar or identical material that has or may come to Sponsor, or any of them, from other sources. Except where prohibited by applicable law, the Entrant acknowledges and agrees that Qualcomm entities, including Sponsor or any of the Released Parties (defined hereafter) do not now and shall not have in the future any duty or liability, direct or indirect, vicarious, contributory, or otherwise, with respect to the infringement or protection of the Entrant’s Intellectual Property Rights in and to the Entrant Materials. The Entrant acknowledges that, with respect to any claim

of the Entrant relating to or arising out of Qualcomm entities', including Sponsor's actual or alleged exploitation or use of any Entrant Material submitted, created or arising in connection with the Hackathon, the damage, if any, thereby caused to the Entrant will not be irreparable or otherwise sufficient to entitle the Entrant to seek any injunctive or other equitable relief or in any way enjoin, otherwise interfere with, delay, or interrupt the production, distribution, exhibition, or other exploitation of any production based on, or allegedly based on the Entrant material, and the Entrant's rights and remedies in any such event shall be strictly limited to the right to recover damages, if any, in an action at law.

- f. **Restriction on Use of Sponsor Marks.** Without the express prior written consent of Sponsor, Entrants shall not make use of any trademark, trade name, brands, logos or other identifying or associated marks of Sponsor or its affiliates, or publicize or disclose the existence or terms of this agreement to any third party by any method including but not limited to any news release, advertisement, publicity or promotional material.

9. **Warranties and Representations.** By registering the Hackathon, you represent and warrant that:
- a. you have reached the age of the majority of the country where the Hackathon occurs and have the full authority and capacity to enter into and be bound by this agreement / Official Rules;
 - b. you are under no contractual obligation or other restriction which is inconsistent with or in conflict with your required performance of your obligations under this agreement / Official Rules or which will interfere with the execution of Submission and/or implementation for Final Submission, and Participant shall not take on any assignments or engage in any activities which is likely to cause a conflict of interest in relation thereto;
 - c. you are not (by virtue of any of your acts and omissions), create any circumstances, obligations or expectations which a third party may use as a basis to make or hold Sponsor or its affiliates liable for any claims relating to, or arising from, (i) your participation in the Hackathon; (ii) your use of the Facilities or QTI Products; or (iii) your use of any products and/or services in connection with the Hackathon that are not provided by Sponsor or its affiliates;
 - d. you agree to be fully responsible for any damage, loss or liability caused as a result of any of your acts or omissions in connection with the Hackathon, including due to any negligence. Upon request from Sponsor or its affiliates, you agree to replace, repair or pay for any damaged or lost equipment, components/parts or other QTI Products supplied by Sponsor or its authorized representatives;
 - e. you agree to be in full compliance of (i) all applicable domestic and international laws and regulations; (ii) Sponsor's policies and procedures provided or made available to you, including without limitation those relating to codes of conduct, ethical behavior, Internet connectivity, security, export control, and the protection and use of intellectual property. Sponsor shall be entitled to require replacement or removal of any person from Facilities for any reason at its sole discretion, including without limitation, misconduct or violation of policies or procedures; and (iii) all applicable terms of use and license terms and conditions applicable to QTI Products. Such use of QTI Products may be required to be

- undertaken solely within the presence of certain designated Sponsor personnel, at Sponsor or its affiliates' discretion;
- f. except for QTI Products, all Entrant Materials are the original works created solely by Entrant or have been properly licensed from the rights owner(s), and not violating or infringing any third party rights, including without limitation, third party Intellectual Property Rights;
 - g. in the event any Entrant Materials rely on, incorporate or otherwise uses any third party material, including music, trademarks, logos, or any copyrighted material, the Entrant shall disclose all details regarding such third party materials to Sponsor at the time of submission and shall ensure it has obtained all of the rights, licenses, permissions and releases in writing from any such third party for use of the material in connection with the Entrant Materials and for the purposes of the Hackathon;
 - h. the Entrant Materials shall comply with these Official Rules and all criteria and guidelines prescribed or otherwise promulgated by or on behalf of Sponsor and shall not violate any applicable law or regulation ;
 - i. Sponsor will not be required to pay or incur any sums to any person or entity in connection with the Entrant Materials;
 - j. the Entrant Materials do not defame, misrepresent, or contain disparaging remarks or any other content which could adversely affect the name, reputation or goodwill of Sponsor, its affiliates, or any other individuals and/or entities;
 - k. Entrant Materials do not contain pornographic or obscene content, hateful content of any kind, content which promotes violence or harm to another living creature, or any other offensive or inappropriate content;
 - l. the Entrant Materials do not include threats of any kind or any content that may intimidate, harass, or bully anyone;
 - m. the Entrant Materials do not contain any viruses, Trojan horses, worms, spyware or other disabling devices or harmful or malicious code.
10. Publicity Release. Entrants agree that Sponsor and its affiliates, contractors and agents shall have the right to promote Entrant Materials but without obligations to do so, and use of the name, location, photograph, graphic, logo, likeness, voice and image of all individuals contributing to Entrant Materials, in any materials promoting or publicizing the Hackathon and its results, during the Hackathon and for three (3) years following the Winner Announcement, throughout the world in any and all media without additional compensation, notification, release, or permission. Sponsor may request that Entrants participate in promotional activities, case studies and/or other marketing activities in connection with the Hackathon, and Entrants agree to use their best efforts to participate in such activities and case studies and agree to comply with all applicable sponsorship and disclosure laws and regulations when participating in any of such activities. Entrants irrevocably consent to and authorize Sponsor and its agents to film, videotape, digitally record and/or photograph Entrants in connection with the Hackathon (the “**Recordings**”). Sponsor and its agents may edit and/or include Recordings, in its sole discretion, in one or more visual, audio and/or audio-visual programs or programming segments prepared by or on behalf of Sponsor and/or its affiliates (“**Programming**”) and to use, reproduce, distribute, exhibit, perform and otherwise exploit any Recordings and/or any and all portions of Recordings in any and all

media whether now known or hereinafter devised throughout the world, in perpetuity and for any purpose or use whatsoever as designated by Sponsor and/or its affiliates. Entrants understand and agree that Entrants shall not be entitled to any compensation with respect to the Recordings or any Programming. Entrants hereby expressly waive and release each of the Sponsor, its affiliates, agents and authorized representatives, and each of its employees, successors, assigns, licensors and licensees from and against any and all claims which Entrants may have relating to invasion of privacy (including personal data), publicity, defamation, infringement of copyright (including moral rights) or any other claim or cause of action arising out of (a) the production, use, distribution, exhibition, advertising, promotion or other exploitation of any Programming, and (b) any and all portions of the Recordings, whether or not included in any Programming.

11. Disclaimer. Sponsor may, at its discretion, provide consultation or advice to you in connection with the development of the working prototype or otherwise in relation to the Hackathon. You acknowledge that Sponsor does not guarantee that access to such personnel will be given and Sponsor makes no assurances that any such granted access will continue or that specific personnel will be made available to you. You further acknowledge that any advice or consultation provided by Sponsor's personnel is for informational purposes only and does not and will not ensure any specific result in the Hackathon. Any and all Facilities, QTI Products, and advice and consultation related to the Hackathon provided by Sponsor or its affiliates, agents or authorized representatives (collectively, "**QTI Deliverables**") are provided solely on a non-exclusive basis to you. You acknowledge and agree that Sponsor or its authorized representatives may simultaneously provide the same or equivalent QTI Deliverables to others, including to other Entrants involved in this Hackathon. ALL QTI DELIVERABLES ARE PROVIDED ON AN "AS-IS" BASIS WITHOUT ANY REPRESENTATIONS OR WARRANTIES OF ANY KIND. YOU ACKNOWLEDGE THAT SPONSOR DOES NOT IN ANY MANNER REPRESENT, WARRANT OR GUARANTEE THAT ANY OF THE QTI DELIVERABLES ARE ERROR-FREE OR WILL CONTRIBUTE TO THE GROWTH OR SUCCESS OF THE BUSINESS OF YOU IN ANY MANNER OR RESULT IN ANY ASSURED OUTPUTS FOR THE FINAL SUBMISSION. IN ADDITION, YOU AGREE AND ACKNOWLEDGE THAT IT IS ONLY PERMITTED TO ACCESS AND USE THE FACILITIES FOR A LIMITED PERIOD OF TIME AS QTI DESIGNATES TO FACILITATE THE FINAL SUBMISSION, AND SUCH ACCESS DOES NOT IN ANY MANNER CREATE, INDICATE, GRANT, VEST OR PROVIDE ANY RIGHTS OR INTERESTS, INCLUDING ANY RIGHT OF ACCESS OR CONTINUED USE, WITH RESPECT TO ANY OF SUCH FACILITIES, INCLUDING THE CREATION OF ANY TENANCY OR ANY OTHER FORM OF PROPERTY RIGHT.
12. Liability Release. Each Entrant agrees that, to the maximum extent permitted by applicable laws, officers, directors, and employees of Sponsor or its parent, subsidiary, and affiliated companies, and each of their respective officers, directors, employees or contractors involved in the Hackathon (collectively "**Released Parties**"): (i) shall not be responsible for or liable for, and are hereby released from any and all costs, injuries, losses or damages of any kind in connection with the Hackathon or otherwise, including without limitation, death and/or bodily injury, due in whole or in part, directly or indirectly, incurred, sustained, arising out of or in connection with the participation in or travel to and from the Hackathon or any Hackathon-related and/or post-

Hackathon activity; or from Entrants' acceptance, receipt, possession and/or use or misuse of any QTI Products, Platforms, Awards, or other item of value from the Released Parties; (ii) assume no responsibility for any damage to any Entrant's, or any other person's, computer system which is occasioned by any websites, platforms, tools, or resources associated with Sponsor and/or its affiliates, or for any computer system, phone line, hardware, software or program malfunctions, or other errors, failures, delayed computer transmissions or network connections that are human or technical in nature; (iii) are not responsible for incomplete, illegible, misdirected, misprinted, late, lost, damaged, or stolen Entrant Material; (iii) are not responsible for lost, inaccessible or unavailable networks, servers, Satellites, Internet service providers, websites, or other connection; (iv) are not responsible for miscommunications, failed, jumbled, scrambled, delayed, or misdirected computer, telephone or cable transmissions; (v) are not responsible for any technical malfunctions, failures, difficulties or other errors of any kind or nature; (vi) are not responsible for the incorrect or inaccurate capture of information, or the failure to capture any information. For purposes of these Official Rules, receipt of Submissions/Final Submissions occurs when Sponsor or its affiliates' website servers successfully receive the uploaded Submission/Final Submission in accordance with guidelines provided in these Official Rules; and (vii) SHALL NOT BE RESPONSIBLE FOR ANY LOSS OF PROFIT, LOSS OF REVENUE, LOSS OF BUSINESS, LOSS OF GOODWILL, OR ANY SPECIAL, INCIDENTAL, CONSEQUENTIAL, PUNITIVE OR OTHER INDIRECT DAMAGES, HOWSOEVER CAUSED, ARISING OUT OF, RESULTING FROM OR IN CONNECTION WITH THIS AGREEMENT AND WHETHER OR NOT ADVISED OF THE POSSIBILITY OF SUCH LOSSES OR DAMAGES. Under no circumstances shall the total aggregate liability of Sponsor and its affiliates under this agreement / Official Rules, whether based in contract, tort, strict liability or upon any other theory of liability, exceed the amount of USD one hundred (\$100.00) dollars. Without prejudice to the foregoing, in no event shall the Released Parties be responsible for any Entrant Materials provided by the Entrant, which shall be the sole responsibility of the Entrant.

13. Indemnification. Notwithstanding the review by Sponsor or any affiliates or the Judges of any Entrant Materials, and to the maximum extent permitted by applicable laws, Entrants understand and agree to bear sole liability and responsibility for the content of Entrant Material and agree to indemnify, hold harmless, and defend the Released Parties for any damages and/or costs (including attorney's fees) arising from or related to: (i) any breach or alleged breach by Entrant of this agreement / Official Rules, including any obligation, covenant, representation or warranty; (ii) any non-compliance with or violation of applicable laws or regulations related to its participation in the Hackathon; (iii) any negligence, recklessness, acts, omissions or misconduct of Entrant related to its participation in the Hackathon; (iv) death or personal injury or property damage arising out of acts or omissions of Entrant related to the Hackathon; and/or (v) a third party's claim or demand relating to the content or any component of Entrant Materials, including, but not limited to, a claim of infringement by such Entrant Material, of a third party's intellectual property rights or privacy rights.
14. Personal Data and Privacy. By registering the Hackathon, each Entrant agrees and consents to Sponsor and its affiliates, contractors, agents and/or assigns collecting, using, and sharing the personal information submitted by the Entrant, including name, address and contact information,

for the purposes of the Hackathon, including administering and conducting the Hackathon and related promotions. Entrants agree to be contacted using the contact information provided. Personal data collected by Sponsor and/or its contractors/agents will be handled in accordance with the privacy policy available on <https://www.qualcomm.com/site/privacy>. Sponsor or its affiliates, contractors, agents and/or assigns will not be responsible or liable in any manner whatsoever for any acts or omissions or any related obligations or liabilities of any third party in relation to any data or information of any Entrants.

15. Modification; Disqualification; Termination.

- a. To the extent permitted by applicable law, Sponsor reserves the right to modify, extend, suspend, or terminate the Hackathon at any time for any reason or no reason, by giving Entrant prior written notice thereof.
- b. Sponsor reserves the right to disqualify any Entrant at any time during the Hackathon, who, at Sponsor's sole discretion, has been determined to have violated the spirit of the Hackathon or breached any terms hereof, including without limitation, breach of a representation (scientific or otherwise) or warranty as provided under these Official Rules; if Entrant shows no material progress or materially deviates from its original Submission without Sponsor's explicit written consent; tampering with the submission process or the operation of the Hackathon, the Hackathon Site, or the Platforms; acting in violation of these Official Rules; acting in an unsportsman-like or disruptive manner; acting with the intent to disrupt or undermine the legitimate operation of the Hackathon; or acting with the intent or outcome to annoy, abuse, threaten or harass any other person, and that Sponsor reserves the right to seek damages and other remedies from any such person to the fullest extent permitted by law. Each Entrant hereby acknowledges that it shall have no right to object to any decision of disqualification taken by Sponsor, which shall be final and binding.
- c. In the event that Entrant is in breach or alleged breach of its obligations under this agreement / Official Rules, Entrant (including all Eligible Individuals in a Team) may be disqualified, at Sponsor's sole discretion, and this agreement / Official Rules may be terminated immediately as of provision of notice from Sponsor through its representative(s).
- d. In the event of any termination of this agreement / Official Rules or disqualification from the Hackathon, Entrant shall (i) immediately cease use and possession of all QTI Products. In such case, Entrant shall first delete or uninstall all QTI Products permanently (including any documentation, binaries, source codes), and return all QTI Products to Sponsor or its authorized representative on site or through shipment within five (5) days; and (ii) promptly remove its equipment and other materials from the Facilities or Hackathon venue, and shall have no further right to access or enter the Facilities or Hackathon venue without Sponsor's explicit consent.
- e. All terms and conditions of this agreement / Official Rules which, by their nature or express terms, are intended to survive expiration or termination of this agreement / Official Rules shall so survive (including, without limitation, terms relating to the protection of Intellectual Property Rights, representations and warranties, and any

provisions relating to the interpretation or enforcement of the terms of this agreement/ Official Rules).

16. Compliance with Laws. Entrants shall at all times comply with all applicable domestic and international laws in connection with their participation in the Hackathon, including but not limited to applicable domestic and international anti-corruption and record-keeping laws and regulations, such as the U.S. Foreign Corrupt Practices Act, and the U.K. Bribery Act, and not directly or indirectly, offer, pay, promise make, offer, authorize, or promise any payments or contribution of money, gifts, business courtesies, hospitality or anything of value to any person for the purpose of influencing any official act or decision of such person in their official capacity, inducing such person to perform or omit any act in violation of their lawful duty, improperly obtain or retain business, securing any improper advantage, pay for improper advantage already secured, or inducing such person to use their influence to affect or influence any act or decision of any foreign or domestic government official or employee, government instrumentality, political party, political official, candidate for political office, member of a royal family, or any officer or employee of a public international organization. The Entrant will strictly comply with all domestica and international export control and economic sanctions laws, orders, and regulations, including but not limited to the Export Administration Regulation, 15 CFR Parts 730-774, the Foreign Assets Control Regulation, 31 CFR Parts 500-599. Subject to relevant laws, Entrant agrees to notify Sponsor of any violation of this section that has or may have occurred. If permitted access to any Sponsor or its affiliates' facilities or infrastructure or systems, Entrants shall comply with Sponsor and its affiliates' policies, regulations and codes of conduct as communicated to them from time to time. When accessing or using Sponsor and its affiliates' websites, including Hackathon Site, Entrants shall be responsible for complying with any policies or legal terms on these websites. Shortlisted Entrants shall be solely responsible for complying with any guidelines, rules, or policies provided by the venue and Sponsor disclaims any liability of Shortlisted Entrants' behavior at the venue and has the discretion to disqualify any Shortlisted Entrants from Phase 2 if Shortlisted Entrants was incompliant with any rules or policies from the venue.

17. Confidentiality.

- a. **“Confidential Information”** means any information of a sensitive, confidential, proprietary nature or otherwise relating to Sponsor or its affiliates, including but not limited to business and/or technical information with respect to operations, business plans, products, roadmaps, strategies, developments, customers, and/or intellectual property, QTI Products, Facilities, and any information or details relating to them (whether or not recorded in documentary form or on computer disk or otherwise recorded or retained) and whether or not such information is expressly stated to be confidential, which is obtained and/or received in connection with the scope of this agreement / Official Rules.
- b. Entrants acknowledge and agree that Submissions, Final Submissions, and any Entrant Material will not be treated as Confidential Information and may be made available to the public, including without limitation, posting on the Internet and social media platforms of

- Sponsor or its affiliates. Sponsor does not guarantee any confidentiality with respect to any Submission, Final Submission, or Entrant Material; and Entrants shall not contain any confidential information or trade secrets into the Submission, Final Submission, or Entrant Material provided to Sponsor.
- c. Entrants shall treat and retain any and all communications or information regarding the Hackathon, including but not limited to status/progress of the Hackathon, shortlists, stages of evaluation, eliminations, as completely confidential and shall not share or disclose any such information with any third party such as fellow competitors or media, on social media, or to others without the prior written consent of Sponsor. Entrants agree that any breach of confidentiality obligations may result in its disqualification from the Hackathon or other measures as determined by Sponsor.
 - d. Entrants shall treat all Confidential Information as confidential, whether or not so identified, and shall not disclose, or permit the disclosure of such information without the prior written consent of Sponsor.
 - e. The terms of this agreement / Official Rules are in addition to the terms of any separate nondisclosure agreement in effect between Entrant and Sponsor, and in the event of any inconsistency between the terms of such agreements, those terms which are most protective of the Confidential Information shall prevail. The confidentiality obligations herein shall survive any expiration or termination of this agreement/ Official Rules.

18. Miscellaneous.

- a. Language Requirements. During Phase 1 and Phase 2, all Entrant Materials must be submitted in English or, if not in English, the Entrant must provide an English translation of the demonstration video, text description, and testing instructions as well as all other materials submitted.
- b. Sponsor Materials. Throughout this Hackathon, Sponsor and/or its affiliates may elect (in its sole discretion and without any obligation to do so) to provide information, materials, products and/or services to Entrants on an as-is basis, solely for use in connection with the development of the Submission and/or Final Submission, to the specific exclusion of any other use, subject to applicable terms and conditions and execution of appropriate agreements by the Entrants.
- c. Enforcement. A person who or which is not a Party to this agreement / Official Rules shall not have any right to enforce any provision of this agreement/ Official Rules.
- d. No Waiver. No waiver, delay or failure by either Party in enforcing any provision of this agreement/ Official Rules shall prejudice or restrict the rights of such Party under this agreement / Official Rules, nor shall a waiver by either Party of any breach operate as a waiver of any subsequent breach. The rights, powers and remedies provided to each Party in this agreement/ Official Rules are in addition to, and do not exclude or limit, any right, power or remedy provided by law.
- e. Severability. If any provision or part-provision of these Official Rules is or becomes invalid, illegal, or unenforceable, it shall be deemed deleted, but that shall not affect the validity and enforceability of the rest of these Official Rules.

- f. No Assignment. Entrant shall not assign any of its rights or obligations under this agreement / Official Rules without the express prior written consent of Sponsor; any attempted assignment in violation of the foregoing shall be void. Sponsor may assign this agreement/ Official Rules without consent from Entrant. This agreement / Official Rules shall be binding upon and inure to the benefit of Sponsor and its successors and permitted assigns.
- g. Notice. Any notice required to be given to Entrant under this agreement / Official Rules must be in writing and sent by email to the email address specified during the registration process. Any notice required to be given to Sponsor under this agreement / Official Rules must be in writing and sent either by (i) prepaid registered post or courier to the Sponsor's mailing address specified below, or (ii) by email to the address specified below.

Notices to QTI:

Qualcomm Technologies, Inc.
5775 Morehouse Drive
San Diego, CA 92121
Attention: Division Counsel
Email: QualcommDeveloper@qti.qualcomm.com

with copy sent to Qualcomm Incorporated:

Qualcomm Incorporated
Attention: Qualcomm Legal Department
5775 Morehouse Drive
San Diego, CA 92121
Email: marketinglegal@qualcomm.com

- h. Governing Law and Venue. This Hackathon is governed by the laws of California, the United States. The competent courts at San Diego, California shall have sole and exclusive jurisdiction over any and all legal actions or claims arising out of, related to or in connection with this Hackathon and these Official Rules.
- i. No Relationship. The Entrant hereby acknowledges and agrees that the relationship between the Entrant and Sponsor (and its affiliates) is not a confidential, fiduciary, exclusive or other special relationship, and the Entrant's decision to submit a Submission, Final Submission, or any Entrant Materials in connection with this Hackathon does not place Sponsor or its affiliates in a position that is any different from the position held by members of the general public with regard to elements of the Entrant Materials, other than as set forth in these Official Rules.
- j. Force Majeure. Sponsor shall not be liable to Entrants for any delay or failure to fulfill its obligations under these Official Rules where any such delay or failure is caused due to any reason beyond the control of Sponsor. In the event an insufficient number of eligible Submissions are received and/or Sponsor is prevented from awarding Awards or

continuing with the Hackathon or by any event beyond its control, including, but not limited to, fire, flood, earthquake, windstorm, epidemic, pandemic, public health crisis, disease, virus, sickness or outbreak or other similar or dissimilar natural disaster; act of God or public enemy; riot or civil disturbance, act of any government or governmental authority, power failure, satellite or equipment failure, failure of telecommunications lines or failure or breakdown of plant, machinery or vehicles operated by a third part; labor dispute, strike, or lockout; actual or threat of war, armed conflict, terrorist attacks, war (declared, underclared or threatened, explosion, nuclear, or chemical or biological contamination; or a government imposed mandate of any kind, order of any court or jurisdiction, or other cause not reasonably within Sponsor's control such as venue shut down (each, a "**Force Majeure**" event or occurrence), Sponsor reserves the right to suspend or terminate the Hackathon at its sole discretion.

- k. Entire Agreement. This agreement constitutes the entire agreement between the Parties and supersedes all prior oral or written negotiations and agreements between the Parties with respect to the subject matter hereof.

Annexure A – Definition

Capitalized and/or italicized terms used in these Official Rules are defined below or being defined throughout the Official Rules.

- a. A “**Restricted Person**” is defined as any legal or natural person (i) listed on a prohibited or restricted party list published by the U.S. government, including but not limited to the U.S. Department of Treasury’s “List of Specially Designated Nationals and Blocked Persons” and “Consolidated Sanctions List”, and the U.S. Department of Commerce’s Entity List, Unverified List, and Denied Persons List, or any similar list maintained by the United Kingdom, the European Union or its Member States, or other applicable local authority; (ii) located, organized or resident in a country or region against which the U.S. government has imposed comprehensive economic sanctions or an embargo; this currently includes Cuba, Iran, North Korea, Syria, and the Crimea and so-called Donetsk People’s Republic and Luhansk People’s Republic regions of Ukraine, but may be amended over time; (iii) owned (50% or more in the aggregate) or controlled, directly or indirectly, by a person or entity described in (i) or (ii) herein; or (iv) otherwise the target of U.S. sanctions (collectively, “**Restricted Persons**”).
- b. “**Agents**” include individuals or organizations are acting on behalf of, and at the direction of, a Promotion Entity through a contractual or similar relationship.
- c. “**The members of Individual’s Immediate Family**” include the individual’s spouse, children and stepchildren, parents and stepparents, and siblings and stepsiblings.
- d. “**The members of an individual’s household**” include any other person that shares the same residence as the individual for at least three (3) months out of the year.
- e. An “**Affiliate**” is: (a) an organization that is under common control, sharing a common majority or controlling owner, or common management; or (b) an organization that has a substantial ownership in, or is substantially owned by the other organization.

Overall Guide

Snapdragon Multiverse Hackathon: Integrated Developer Guide

Welcome to the Snapdragon Multiverse Hackathon. This guide provides a unified technical workflow for deploying Generative AI and Classical ML models using the Qualcomm ecosystem, specifically focusing on the LiteRT and LiteRT-LM frameworks on the Samsung Galaxy S25 Ultra.

Event Overview & Target Hardware

The hackathon aims to develop AI experiences across multiple form factors including AI PCs (Snapdragon X Series), mobile devices, and microcontrollers like the Arduino UNO Q.

Primary Device Specifications

- Device: OnePlus 15
- Chipset: Snapdragon® 8 Elite (SM8850).
- Acceleration: Hexagon NPU (v81) for high-efficiency hardware acceleration.
- Compute : Snapdragon X-elite
- IoT: Arduino UnoQ

Core Architectural Frameworks

The hackathon is divided into two primary tracks based on the model type and required runtime.

Runtime	Primary Use Case	Model Format
LiteRT-LM	Large Language Models (LLMs) and Vision-Language Models (VLMs).	<code>.litemlm</code> .
LiteRT	Classical ML (Classification, Segmentation, Object Detection).	<code>.tflite</code> .
ExecuTorch	Cross-platform Edge AI	Executable
llama.cpp with Qualcomm Backend	LLM Inference	GGUF
Arduino UNOQ	Microcontroller ML	Native
EdgeImpulse	ML Pipeline	Various
AI Hub	Optimization & Deployment	Various

Backend Stack Hierarchy

For maximum performance on Snapdragon platforms, utilize the NPU (QNN) backend.

- NPU (QNN): High-efficiency acceleration for Snapdragon 8 Elite (SM8750).
- GPU: Parallel task acceleration for standard Android performance boosts.
- CPU: Universal fallback for all devices.

Submission & Evaluation

Project Requirements

To be eligible for prizes, submissions must include:

- GitHub Repository: Containing open-source code and a detailed README.
- README Components: Description, names/emails of team members, setup instructions, and an open-source license.
- Edge Execution: The application and most components must run locally on the device.

Judging Rubric (Scored 1-10)

1. Technical Implementation (40%): Performance, latency, and resource utilization on the NPU.
2. Innovation (25%): Uniqueness and real-world problem solving.
3. Deployment (20%): Ease of installation and stability during demos.
4. Presentation (15%): Clarity and code quality.

Key Developer Resources

- Qualcomm AI Hub: aihub.qualcomm.com.
- LiteRT Samples: github.com/google-ai-edge/litert-samples.
- Hugging Face (LiteRT Community): Pre-quantized models for Gemma and other architectures.
- Support: Real-time assistance available on the Qualcomm Discord.

LiteRT

Qualcomm Hackathon: LiteRT Developer Guide

This technical guide provides a comprehensive workflow for developers deploying the Generative AI and Classical ML models using LiteRT and LiteRT-LM.

Table of Contents

[Google x Qualcomm Hackathon: Developer Guide](#)

[Pre-work](#)

[Core Architectural Concepts](#)

[LiteRT vs. LiteRT-LM](#)

[Backend Stack Hierarchy](#)

[Hackathon Tracks](#)

[Models](#)

[Additional References](#)

- [Device Details](#)
- [Track 1: LiteRT-LM for Generative AI \(e.g. Gemma3 1B & FastVLM\)](#)
- [Track 2: LiteRT for Classical Models](#)
- [Details for Leverage Google AI Edge Gallery App](#)

Pre-work

Suggest participants try out the samples below to get acquainted with LiteRT and LiteRT-LM

1. Try out the Qualcomm LiteRT & LiteRT-LM [sample apps](#) with LiteRT and LiteRT-LM. Additional samples to learn more about LiteRT CPU/GPU/NPU(jit) can be found here: [Sample Apps](#)
 - Models used in the sample apps can be found on the following Hugging Face collection:
<https://huggingface.co/collections/litert-community/qualcomm>
2. Go over the documentation
 - <https://ai.google.dev/edge/litert-lm> for Track 1
 - <https://ai.google.dev/edge/litert> for Track 2

Core Architectural Concepts

Understanding the relationship between the runtime and its specialized wrapper is essential for efficient deployment.

LiteRT vs. LiteRT-LM

- LiteRT: Serves as the foundational high-performance runtime for on-device machine learning, executing .tflite models across various hardware.
- LiteRT-LM: A specialized high-level API wrapper powered by LiteRT specifically for Large Language Models (LLMs) and Vision-Language Models (VLMs). It automates complex tasks such as tokenization, KV-caching, and multimodal input handling. LiteRT-LM executes .litemlm files.

Backend Stack Hierarchy

LiteRT optimizes performance by offloading computation to available hardware:

Backend	Capability	Optimal Use Case
CPU	Universal fallback for all devices	Low-complexity models or non-accelerated hardware.
GPU	Parallel task acceleration	Standard Android performance boost.
NPU (QNN)	High-efficiency hardware acceleration	Snapdragon 8 Elite (SM8750) for maximum speed.

Track 1: LLM-Based Consumer Use Journeys with LiteRT-LM

Build on-device LLM experiences using Generative AI models with a focus on real consumer use cases.

What’s provided:

- Ready-to-use the Generative AI models, Gemma, EmbeddingGemma, and FastVLM, available via [LiteRT Hugging Face](#)
- A reference [sample app](#) for Gemma 4 2B model

This track is ideal for developers interested in integrating Generative AI-powered features into Android applications with on-device inference.

Track 2: Classical Models — Vision & Audio with LiteRT

Build efficient, on-device AI applications using classical ML models for vision or audio workloads.

What's provided:

- Ready-to-use models that do not require quantization or conversion
- NPU-friendly models available via [Qualcomm AI Hub](#) and [LiteRT HuggingFace](#)
- A sample application available through the LiteRT repository under the LiteRT-Samples github repo Qualcomm [directory](#) (mobilenet v2, object_detection), [EmbeddingGemma](#), and [image segmentation](#)

This track is ideal for developers looking to deploy performant vision or audio pipelines on Snapdragon NPUs with minimal overhead.

Models

<https://huggingface.co/collections/litert-community/qualcomm>

Track 1: Generative Models via LiteRT-LM

[Gemma4 E2B](#) ([Sample app for NPU](#) / [Sample app for CPU/GPU](#))

Gemma 4 E2B is a multimodal "edge" model released by Google DeepMind in April 2026. The "E" stands for Effective parameters, meaning it uses a specialized architecture to achieve high performance with a low memory footprint suitable for local, on-device deployment.

- gemma-4-E2B-it_qualcomm_sm8750.litertlm: Specifically compiled for the Snapdragon 8 Elite NPU.

- gemma-4-E2B-it.litertlm: Designed for standard GPU and CPU acceleration.

[FastVLM-0.5B](#) (Sample app for CPU/GPU/NPU)

FastVLM is a multimodal vision-language model capable of interpreting image data and generating text responses.

- FastVLM-0.5B.qualcomm.sm8750.litertlm Specifically compiled for the Snapdragon 8 Elite NPU.

[Gemma3 1B](#)

Gemma 3 1B is a lightweight, open-weight language model from Google, optimized for on-device, mobile, and web applications. Released in March 2025, this 1-billion-parameter model offers high efficiency for text-based tasks.

- Gemma3-1B-IT_q4_ekv1280_sm8750.litertlm Specifically compiled for the Snapdragon 8 Elite NPU.
- Gemma3-1b-it-int4.litertlm Designed for standard GPU and CPU acceleration.

[Function Gemma](#)

FunctionGemma is a specialized 270M parameter model based on Gemma 3, fine-tuned for efficient function calling and API action generation in local, on-device AI applications. It converts natural language into structured function calls, providing high performance for mobile/IoT devices with low latency. (Gallery App [mobileactions](#), [tinygarden](#))

- Mobile_actions_q8_ekv1024.litertlm Designed for standard GPU and CPU acceleration.

Track2: LiteRT, Vision and Audio (aka: traditional/classical models)

For more models: <https://aihub.qualcomm.com/apps?os=Android>

Models / Demo Page	CPU/GPU/NPU (JIT)	Sample App
EmbeddingGemma-300m	Supported	Sample App
Image Segmentation	Supported	Sample App (CPU/GPU) Sample App (NPU)
Image Classification	Supported	Sample App
Object Detection	Supported	Sample App

Note: Migrating guide from TensorFlow Lite to LiteRT - [link](#)

Additional References

Applicable if you are not starting with a sample app.

Device Details

- Type: Mobile
- Name: One Plus 15
- Chipset: Snapdragon® 8 Elite Gen 5
- Hexagon Version and SOC Model: v81 and 69, respectively

Track 1: LiteRT-LM for Generative AI (e.g, Gemma4 E2B & FastVLM)

This guide details the LiteRT-LM ([repo](#)) Developer Workflow for implementing machine learning inference in Android applications using the CompiledModel API. The following documentation provides instructions and code snippets for setting up and running models using various hardware accelerators, including CPU, GPU, and NPU, and outlines the core steps for creating an environment, configuring options, and performing inference.

The development workflow can be separated into the following two major entry points: 1) Use our Kotlin API with Maven package; 2) Use our C++ API and build from

source

Kotlin API with Maven Package

Please follow the [instructions](#) in the official documentation to set up your project for importing the LiteRT-LM Maven package.

To run the models on NPU, please make sure you copy the NPU libraries from [the sample app](#) to your app folder. The required libraries are:

- `libLiteRtDispatch_Qualcomm.so`
- `libQnnHtp.so`
- `libQnnHtpV81Skel.so`
- `libQnnHtpV81Stub.so`
- `libQnnSystem.so`

Separately, you'd also need to copy the prebuilt `libGemmaModelConstraintProvider.so` from the [LiteRT-LM/prebuilt](#) folder (from the corresponding platform) to your app folder.

This [repo](#) gives you an example sample app that uses the Gemma4 model with NPU.

1. Setting Up the Engine

The *Engine* is the brain that runs your model. You need to provide a configuration that tells it where your model file is located and which hardware (like the NPU for speed) to use for both text and images.

```
Kotlin
val config = EngineConfig(
    modelPath = "/path/to/gemma-4-E2B-it_qualcomm_sm8750.litertlm",
    backend = Backend.NPU(nativeLibDir), // Use NPU for text
    visionBackend = Backend.GPU(),       // Use GPU for images
    audioBackend = Backend.CPU()         // Use CPU for audio
)
val engine = Engine(config)
engine.initialize()
```

2. Creating a Conversation

A *Conversation* object helps the AI remember what was said earlier in the chat. You create it directly from your initialized engine.

Kotlin

```
val conversation = engine.createConversation()
```

3. Sending Messages with Images

Finally, you can send a message that includes both an image and a text question. The AI will process these together and send back the response in small pieces (chunks) that you can print or show in your app.

Kotlin

```
val userMessage = Message.user(
    Contents.of(
        Content.ImageFile("/path/to/photo.jpg"),
        Content.Text("What is in this image?")
    )
)
conversation.sendMessageAsync(userMessage).collect { response ->
    val chunk = response.contents.contents
        .filterIsInstance<Content.Text>()
        .joinToString("") { it.text }
    println(chunk)
}
```

C++ API and Build from Source

Please follow the [instructions](#) in the official documentation to set up the environment and build the program. Next, use the following code snippets to start playing with it.

C/C++

```
#include "runtime/engine/engine.h"

// ...
```

```

// 1. Define model assets and engine settings.
auto model_assets = ModelAssets::Create(model_path);
CHECK_OK(model_assets);

auto engine_settings = EngineSettings::CreateDefault(
    model_assets,
    /*backend=*/litert::lm::Backend::CPU);

// 2. Create the main Engine object.
absl::StatusOr<std::unique_ptr<Engine>> engine =
    Engine::CreateEngine(engine_settings);
CHECK_OK(engine);

// 3. Create a Conversation
auto conversation_config =
    ConversationConfig::CreateDefault(**engine);
CHECK_OK(conversation_config)
absl::StatusOr<std::unique_ptr<Conversation>> conversation =
    Conversation::Create(**engine, *conversation_config);
CHECK_OK(conversation);

// 4. Send message to the LLM with blocking call.
absl::StatusOr<Message> model_message = (*conversation)->SendMessage(
    JsonMessage{
        {"role", "user"},
        {"content", "What is the tallest building in the world?"}
    });
CHECK_OK(model_message);

// 5. Print the model message.
std::cout << *model_message << std::endl;

// 6. Send message to the LLM with asynchronous call
// where CreatePrintMessageCallback is a users implemented callback
// that would
// process the message once a chunk of message output is received.
std::stringstream captured_output;

```



```

(*conversation)->SendMessageAsync(
    JsonMessage{
        {"role", "user"},
        {"content", "What is the tallest building in the world?"}
    },
    CreatePrintMessageCallback(std::stringstream& captured_output)
);
// Wait until asynchronous finish or timeout.
*engine->WaitUntilDone(absl::Seconds(10));

```

Track 2: LiteRT for Classical Models

(e.g., Image Segmentation)

The LiteRT CompiledModel API is used for smaller, classical machine learning models (like image classification or segmentation) that don't require the specialized features of LiteRT-LM (Type 1). This is the standard way to deploy **.tflite** models.

Using CompiledModel API with CPU/GPU acceleration

This section shows how to use LiteRT's CompiledModel API with CPU and GPU acceleration. It talks about the two most common Android integration paths:

1. Kotlin / Android runtime (com.google.ai.edge.litert:litert)
2. Android NDK / C++ with the prebuilt LiteRT C++ SDK

CompiledModel is a runtime entry for LiteRT. In the current API, the core flow is:

1. Create one shared Environment.
2. Create Options and select CPU or GPU acceleration.
3. Configure backend-specific CpuOptions or GpuOptions if needed.
4. Create a CompiledModel.
5. Create input/output TensorBuffers and run inference.

1. Setup and Dependencies

First, you need an Android project (which requires Android Studio and Kotlin knowledge).

Gradle Dependency

Add the core LiteRT library to your `build.gradle.kts` file. This handles both CPU and GPU acceleration by default for Kotlin users.

```
Kotlin
dependencies {
    // Core LiteRT library
    implementation("com.google.ai.edge.litert:litert:2.1.4")
}
```

2. Android release artifacts to use for v2.1.4

For Android apps using the LiteRT Android binding, use:

```
implementation("com.google.ai.edge.litert:litert:2.1.4")
```

This is pretty much the simplest way to include GPU support on Android. For Kotlin users, the GPU accelerator is already bundled in the LiteRT Maven package.

You can then select CPU vs GPU in Kotlin by using `CompiledModel.Options(...)`, see the following code snippet

```
Kotlin
val env = Environment.create()

val cpuModel =
    CompiledModel.create(
        context.assets,
        "model.tflite",
        CompiledModel.Options(Accelerator.CPU),
        env,
    )

val gpuModel =
    CompiledModel.create(
```

```
context.assets,  
    "model.tflite",  
    CompiledModel.Options(Accelerator.GPU),  
    env,  
)
```

3. Model Loading and Configuration

Models like image segmentation typically use a standard `.tflite` format. You must load this model and select your desired hardware accelerator (CPU, GPU, or NPU).

Model Loading (Kotlin)

```
Kotlin  
  
import com.google.ai.edge.litert.CompiledModel  
import com.google.ai.edge.litert.Accelerator  
  
// 1. Select the accelerator  
val options = CompiledModel.Options.builder()  
    .setAccelerator(Accelerator.GPU) // Choose GPU for a standard  
    speed boost  
    .build()  
  
// 2. Load the model from your app's assets folder  
// The model must be packaged in your project (e.g., in the  
// 'assets' folder).  
val compiledModel = CompiledModel.create(  
    context.assets,  
    "segmentation_model.tflite", // Replace with your model file  
    name  
    options  
)
```

4. Running Inference

The execution flow involves preparing input data, creating buffers, running the model, and reading the results. Crucially, heavy operations like this should always be run off the main

application thread (e.g., using Kotlin Coroutines).

The basic execution flow is the same for CPU and GPU once the model has been Compiled.

CPU / GPU / NPU execution & Running Inference

Kotlin

```
// 1. Load model and initialize runtime.
val model =
    CompiledModel.create(
        context.assets,
        "model/mymodel.tflite",
        CompiledModel.Options(Accelerator.CPU, Accelerator.GPU)
    )

// 2. Pre-allocate input/output buffers
val inputBuffers = model.createInputBuffers()
val outputBuffers = model.createOutputBuffers()

// 3. Fill the first input
inputBuffers[0].writeFloat(...)

// 4. Invoke
model.run(inputBuffers, outputBuffers)

// 5. Read the output
val outputFloatArray = outputBuffers[0].readFloat()
```

Setup with NPU acceleration

To run classic models on NPU with JIT mode, please make sure you copy the NPU libraries from [the sample app](#) to your app folder. The required libraries are:

- **libLiteRtCompilerPlugin_Qualcomm.so**
- **libLiteRtDispatch_Qualcomm.so**
- **libQnnHtp.so**
- **libQnnHtpPrepare.so**
- **libQnnHtpV81Skel.so**
- **libQnnHtpV81Stub.so**
- **libQnnSystem.so**

In production, it's recommended to use Google Play Feature Delivery to ship NPU libraries

only to targeted devices. See <https://ai.google.dev/edge/litert/next/npu> for the instructions.

Create CompiledModel for Inference with NPU

```
Kotlin
val env = Environment.create(BuiltinNpuAcceleratorProvider(context))

// For JIT models, create CompiledModel with model path directly
val jitCompiledModel = CompiledModel.create(
    "/path/to/my_jit_model.tflite",
    CompiledModel.Options(Accelerator.NPU),
    env,
)

// then run inference
```

Details for Leverage Google AI Edge Gallery App

Users can clone Google AI Edge and build their app based on the following instructions.

Clone the Google AI Edge to build your own app

Detailed documentation can be found in the [LiteRT-LM Kotlin README](#), but you can also start by [cloning the Google AI Edge Gallery app code](#) that's built on top of LiteRT-LM Kotlin APIs.

Step 1: Clone the Repository

First, grab the source code from GitHub. Open your terminal and run the following command:

```
Shell
git clone git@github.com:google-ai-edge/gallery.git
```

This will create a local copy of the repository in a directory named gallery.

Step 2: Navigate to the Android Source

The Android project is located within the `Android/src/` directory of the repository. Change your current directory to this location:

```
Shell
cd gallery/Android/src/
```

Step 3: Build and Install

Now, we'll use the Gradle wrapper to build the debug version of the app and install it directly onto your connected device. Run:

```
Shell
./gradlew installDebug
```

Gradle will take care of downloading dependencies, compiling the code, and deploying the APK. Once finished, you should see "Edge Gallery" appearing in your app drawer.

Step 4: Link to Gallery App Gemma1B CPU/GPU/NPU

<https://github.com/google-ai-edge/gallery/blob/main/Android/src/app/src/main/java/com/google/ai/edge/gallery/ui/llmchat/LlmChatModelHelper.kt#L168>

Executorch

ExecuTorch: Developer Guide

Overview

This technical guide provides a comprehensive workflow for developers participating in the Qualcomm x Meta ExecuTorch Hackathon, covering model compilation, Qualcomm backend integration, Android deployment, and device-specific optimization.

Table of Contents

1. Core Architectural Concepts

- Why ExecuTorch?
- PyTorch to ExecuTorch Workflow
- Backend Stack Hierarchy
- ExecuTorch Runtime Overview

2. Qualcomm AI Runtime (QAIRT)

- Workflow
- Setup
- Building ExecuTorch

3. Models & Deployment Strategies

- Model Compilation
- Examples

4. Android Development Workflow

- Build Android Runtime
- Integrate Runtime
- Load Model
- Run Inference
- Testing

5. Additional Resources

Core Architectural Concepts

Why ExecuTorch?

ExecuTorch is PyTorch's on-device inference framework designed to bring AI models from development to deployment across mobile, edge, and embedded devices.

It enables developers to:

- Build and train models in PyTorch
- Export models into an optimized runtime format
- Execute models efficiently on-device

Benefits

- Low memory overhead
- Minimal runtime dependencies
- Responsive AI experiences
- Hardware-specific acceleration
- Local inference execution

For Snapdragon-powered devices, ExecuTorch integrates with Qualcomm's QAI RT API to leverage dedicated AI hardware, enabling:

- Improved performance
- Lower latency
- Greater power efficiency

This allows developers to build applications that benefit from:

- Real-time inference
- Offline operation
- Privacy-preserving processing

Applicable workloads include:

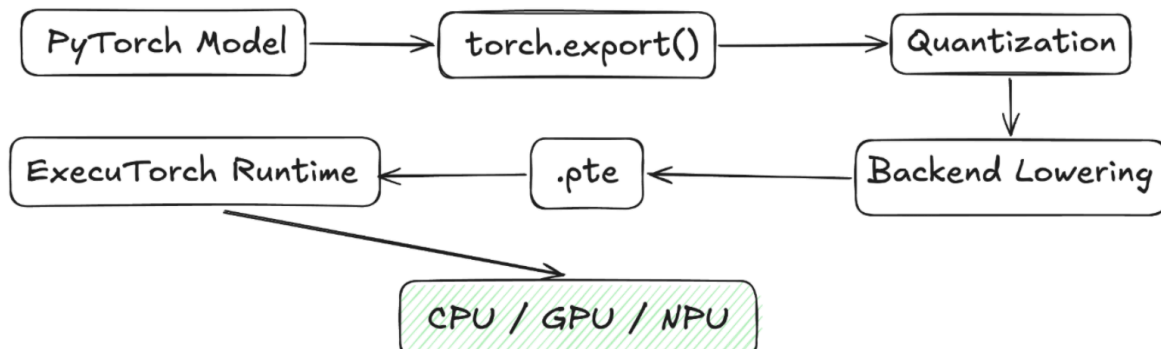
- Computer vision
- Multimodal AI
- Generative AI assistants

ExecuTorch provides a streamlined path from PyTorch model development to optimized deployment on edge devices.

PyTorch to ExecuTorch Workflow

ExecuTorch starts from native PyTorch models and produces hardware-specific `.pte` artifacts that are executed by the ExecuTorch Runtime.

Workflow



Understanding the relationship between export, lowering, and runtime execution is essential to understanding the workflow.

Backend Stack Hierarchy

ExecuTorch supports multiple acceleration backends.

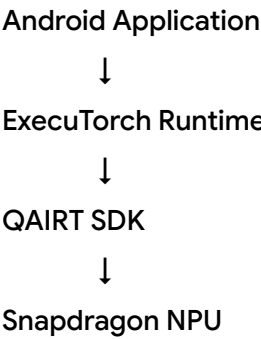
Backend	Capability	Optimal Use Case
XNNPACK CPU	Universal fallback	Maximum compatibility
Vulkan GPU	Parallel acceleration	GPU execution
Qualcomm QNN NPU	Snapdragon acceleration	Highest performance and efficiency

Recommendation

For Snapdragon devices, Qualcomm QNN is the preferred backend.

ExecuTorch Runtime Overview

Recommended architecture:



ExecuTorch x Qualcomm AI Runtime (QAIRT)

Workflow

1. Build AOT components on the host machine.
2. Push compiled models to target device (OnePlus 15).

3. Run inference on the device.

Documentation

<https://docs.pytorch.org/executorch/stable/backends-qualcomm.html>

Setup

Environment Setup

Create and activate a virtual environment.

Clone Repository

```
git clone https://github.com/pytorch/executorch.git  
cd executorch
```

Sync and Initialize Submodules

```
git submodule sync  
git submodule update --init --recursive
```

Install Dependencies

```
pip install -r requirements.txt  
pip install -r requirements-examples.txt  
pip install -e .
```

Optional Downloads

Download QAI RT SDK

Download and install the Qualcomm AI Runtime SDK.

Download Android NDK v26

Download Android NDK version 26.

Configure Environment Variables

```
export QNN_SDK_ROOT=<path_to_qnn_sdk>/2.46.0.260424/
```

```
export ANDROID_NDK_ROOT=<path_to_android_ndk>
```

Important: Android NDK 26 is required.

```
export LD_LIBRARY_PATH=$QNN_SDK_ROOT/lib/<specific tool chain>/:$LD_LIBRARY_PATH
```

Build ExecuTorch

Pathway 1 — Using Your Favorite Coding Agent

Use the following prompt:

Build ExecuTorch with Qualcomm backend support

Pathway 2 — Build Script

```
./backends/qualcomm/scripts/build.sh
```

Models & Deployment Strategies

Compiling Models

Key Arguments

Build Output Folder

`--build_path build-android`

- Created during ExecuTorch build process.

SoC Model

`--soc_model SM8850 (v81)`

- One Plus 15 = SM8850

Model Path

`--encoder_model <model_to_compile>`

Compile Only

`--compile_only`

- Generates only the `.pte` artifact.
-

Examples

GitHub Example Scripts:

https://github.com/pytorch/executorch/tree/main/examples/qualcomm/oss_scripts

Android Development Workflow

Build Android Runtime

```
./scripts/build_android_library.sh
```

Integrate Runtime (Using AAR)

Add the generated AAR to your Android project:

```
implementation(files("libs/executorch.aar"))
```

Load Model

```
Module module =  
    Module.load("MODEL.ptc");
```

Run Inference

```
module.forward(...);
```

Testing

Validate application and model performance using the following metrics:

- Cold start time
 - Inference latency
 - Memory usage
 - Battery impact
-

Additional Resources

ExecuTorch Documentation

<https://docs.pytorch.org/executorch/>

PyTorch Export Documentation

<https://pytorch.org/docs/stable/export.html>

Qualcomm AI Hub

<https://aihub.qualcomm.com/>

ExecuTorch GitHub Repository

<https://github.com/pytorch/executorch>

Device Details

Device Information

Attribute	Value
Type	Mobile
Device Name	One Plus 15
Chipset	Snapdragon® 8 Elite Gen 5
SoC Model	SM8850
Hexagon Version	v81
SOC Version	69

End-to-End Summary

PyTorch Model



`torch.export()`



Quantization



Qualcomm Backend Lowering (QNN)



.pte Artifact



ExecuTorch Runtime



QAI RT SDK



Snapdragon NPU (OnePlus 15)



On-device Inference

Uno Q

Arduino UNO Q: Developer Guide

This technical guide provides a comprehensive workflow for developers building dual-brain edge applications on the Arduino UNO Q, combining a Linux microprocessor (MPU) and a real-time microcontroller (MCU) through Arduino App Lab.

Table of Contents

- Arduino UNO Q Hackathon: Developer Guide
 - Pre-work
 - Core Architectural Concepts
 - MPU vs. MCU
 - The Bridge (RPC) and Router
 - Apps and Bricks
 - Hackathon Tracks
 - Track 1: AI & Vision/Audio Apps (MPU + Bricks)
 - Track 2: Real-Time Hardware & Sensing (MCU + Bridge)
 - Device Details
 - Development Workflows
 - Workflow A: Arduino App Lab (recommended)
 - Workflow B: Arduino IDE / CLI (MCU only)
 - Workflow C: Standalone Linux (SBC mode)
 - Additional References
-

Pre-work

Before the event, participants should get acquainted with the board and Arduino App Lab:

- Download and install Arduino App Lab for your OS from the official page: <https://www.arduino.cc/en/uno-q> (Linux Debian "Bookworm"+/Ubuntu 22.04+, plus macOS and Windows builds). App Lab also comes pre-installed on the UNO Q itself.
- Read the getting-started material:
 - Product page and overview: <https://www.arduino.cc/en/uno-q>
 - Hardware documentation: <https://docs.arduino.cc/hardware/uno-q/>
 - UNO Q User Manual: <https://docs.arduino.cc/tutorials/uno-q/user-manual/>

- App Lab quickstart / tutorial:
<https://docs.arduino.cc/software/app-lab/tutorials/getting-started/>
- Run the built-in Blink LED example (see Workflow A). It is the "Hello World" of the dual-processor architecture: the Python side (MPU) toggles state and logs it, while the sketch (MCU) drives the physical LED — connected through the Bridge.
- Browse the built-in Examples and Bricks inside App Lab to see what's available out of the box.

About Arduino App Lab

Arduino App Lab is the unified development environment and workspace for the UNO Q, designed to seamlessly combine C++ Arduino sketches, Python scripts, and AI models into ready-to-run apps. It handles the orchestrated management of both the Linux-based MPU and the Zephyr-based MCU environments, ensuring tight integration across the dual-brain architecture.

The platform simplifies modular development through the use of Bricks, which allow developers to add complex features like Camera, Object Detection, and Web Servers with ease. Furthermore, the live console provides a centralized interface for real-time debugging and tracing of both MPU and MCU execution logic.

Core Architectural Concepts

Understanding the relationship between the two processors and how they talk to each other is essential for efficient development.

MPU vs. MCU

The UNO Q is a single board carrying two compute domains in the classic UNO form factor:

- MPU (Microprocessor Unit): Qualcomm Dragonwing™ QRB2210 running a full Debian Linux OS. This is where you run Python, AI models, computer vision, audio processing, networking, and web services. Think of it as a Raspberry Pi-class Linux computer on the board.
- MCU (Microcontroller Unit): STMicroelectronics STM32U585 (Arm® Cortex®-M33) running Arduino Sketches over Zephyr OS. This handles deterministic, real-time control: reading sensors, driving actuators, GPIO, and timing-sensitive tasks — the classic Arduino experience in C/C++.

The strength of the platform is combining both: heavy processing on the MPU, precise hardware control on the MCU.

The Bridge (RPC) and Router

The two processors communicate through a built-in RPC (Remote Procedure Call) system, exposed to developers as the Bridge library and managed by the arduino-router service.

- One side registers a function it wants to expose (service discovery); the other side calls it remotely as if it were local.
- Directions supported: MPU → MCU, MCU → MPU, and Linux ↔ Linux (one Linux process calling another).
- Resource lock: the Router exclusively claims the serial interface linking the two chips (e.g. `Serial1` on the Arduino/Zephyr side). Do not access these directly in your own code or the Bridge will fail.
- Linux processes reach the Router via a Unix Domain Socket; advanced users can speak the protocol directly with `msgpack` without the App Lab helper classes.

Apps and Bricks

App: a complete, self-contained project that coordinates both processors. Its standard structure is:

```
my_app/
├── README.md
├── app.yaml
├── python/
│   ├── main.py
│   └── requirements.txt
└── sketch/
    ├── sketch.ino
    └── sketch.yaml
```

- Filenames and directories matter — the UNO Q runtime looks for `python/` and `sketch/` with these exact names.
- Bricks: modular, plug-and-play software components you drop into an App to add ready-made functionality — e.g. Camera, Object Detection, Web Server, and other AI/vision/audio and integration blocks. Bricks are the fastest way to add complex features during a hackathon.

Hackathon Tracks

Participants choose one of the following tracks. Both are built on the same App Lab workflow; they differ in where the "center of gravity" of your project sits.

Track 1: AI & Vision/Audio Apps (MPU + Bricks)

Build intelligent applications that sense and react to their environment using the Linux side and AI Bricks.

What's provided / ideal for:

- Python running on the QRB2210 MPU, with Camera and Object Detection Bricks and other AI models available in App Lab.
- Optional USB camera / MIPI-CSI camera, USB microphone, and a display via USB-C dongle.
- Best for: AI-powered vision, audio classification, smart-home logic, dashboards/web interfaces that also drive physical outputs through the MCU.

Typical shape: Python (MPU) does detection/inference and web/API work → sends results over the Bridge → sketch (MCU) reacts on real hardware (LEDs, motors, relays, displays).

Track 2: Real-Time Hardware & Sensing (MCU + Bridge)

Build responsive, hardware-centric projects where the STM32 MCU does deterministic control and the MPU adds logic, connectivity, or logging.

What's provided / ideal for:

- Full Arduino shield + library compatibility (traditional UNO headers), Qwiic connector for Modulino nodes, and SPI/I²C/UART peripherals.
- Best for: sensor hubs, motor/robotics control, IoT prototypes, data logging, and anything needing tight timing that then hands data up to Linux for storage, networking, or visualization.

Typical shape: Sketch (MCU) reads sensors / controls actuators → exposes or calls RPC functions over the Bridge → Python (MPU) logs, serves a web UI, or pushes data to the cloud.

Device Details

Item	Specification
Type	Single-Board Computer (UNO form factor)
Name	Arduino UNO Q (ABX00162 / ABX00173)
MPU	Qualcomm Dragonwing™ QRB2210 — Quad-core Arm® Cortex®-A53 @ 2.0 GHz, Adreno GPU, 2× ISP (13 MP + 13 MP or 25 MP) @ 30 fps
MPU OS	Debian Linux (upstream support)
MCU	STMicroelectronics STM32U585 — Arm® Cortex®-M33 up to 160 MHz, 2 MB flash, 786 kB SRAM, FPU
MCU OS	Zephyr (runs Arduino sketches)
Wireless	WCBN3536A: Wi-Fi® 5 dual-band (2.4/5 GHz), Bluetooth® 5.1, onboard antenna
Expansion	Qwiic connector (Modulino), UNO headers (shields), bottom high-speed connectors (carriers: MIPI-CSI camera, MIPI-DSI display, analog audio)
RAM variants	2 GB / 4 GB (4 GB recommended for standalone SBC use)
Onboard LEDs	4 RGB user LEDs — #1 & #2 controlled by MPU, #3 & #4 by MCU
Power	USB-C (use a USB-C dongle with Power Delivery for standalone/desktop use — without PD the board will not boot)

Pinout / datasheet / schematics:

- Full pinout (PDF):
<https://docs.arduino.cc/resources/pinouts/ABX00162-full-pinout.pdf>
 - Datasheet:
<https://docs.arduino.cc/resources/datasheets/ABX00162-ABX00173-datasheet.pdf>
 - Schematics:
<https://docs.arduino.cc/resources/schematics/ABX00162-schematics.pdf>
-

Development Workflows

There are three main entry points. App Lab is recommended because it orchestrates both processors together.

Workflow A: Arduino App Lab (recommended)

App Lab is the integrated environment that builds the sketch, sets up the Python environment, installs the Bricks defined in the App, and launches both sides together.

1. Connect and configure

- Connect the UNO Q to your computer via USB-C (or use it standalone with an HDMI display, keyboard, and mouse through a PD dongle).
- On first use, follow the prompts to update the firmware, set a username/password, and configure Wi-Fi. This also enables an SSH server on the board.

2. Navigate the interface

The left sidebar provides: My Apps (your projects), Examples (built-in, read-only), Bricks (modular blocks), Learn (offline docs), and Settings (Wi-Fi, Linux password, keyboard layout).

3. Run a built-in example

- Open Examples → Blink LED and press Run. App Lab compiles the C++ sketch, builds the Python environment, and launches both. The Console panel opens at the bottom; when the Run button turns into Stop and a green notice appears, the App is live. LED3_R (an MCU-controlled LED) begins blinking.

4. Copy and edit

- Built-in examples are read-only. Choose Copy and edit app, name it, and Create new to get an editable copy. Use the File Manager in the sidebar to open `python/main.py` and `sketch/sketch.ino`.

5. Log and debug

- Standard Python `print()` statements are captured automatically and shown in the Python tab of the Console — the simplest way to trace behavior.

Minimal cross-processor example (Bridge)

MCU side ([sketch/sketch.ino](#)) — register a function the Linux side can call:

```
#include <Arduino.h>
#include "Bridge.h" // Arduino Bridge (RPC) library

const int LED_PIN = LED_BUILTIN;

// This function is exposed over RPC and called from Python (MPU)
void set_led_state(bool on) {
    digitalWrite(LED_PIN, on ? HIGH : LOW);
}

void setup() {
    pinMode(LED_PIN, OUTPUT);
    Bridge.begin();
    Bridge.provide("set_led_state", set_led_state); // service discovery
}

void loop() {
    Bridge.update();
}
```

MPU side ([python/main.py](#)) — call the MCU function over the Bridge:

```
import time
from arduino.app_bridge import Bridge # App Lab helper

bridge = Bridge()

state = False
while True:
    state = not state
    bridge.call("set_led_state", state) # remote call to the MCU
    print(f"LED set to {state}")      # shows in the Python console tab
    time.sleep(1)
```

API names above illustrate the pattern (register on one side, call from the other). Use the exact class/method names from the built-in Bridge examples in App Lab and the User Manual, as helper APIs may differ by App Lab version.

Workflow B: Arduino IDE / CLI (MCU only)

If you only need the microcontroller (classic Arduino experience):

- Use Arduino IDE 2.0+ to program the STM32U585 MCU directly in C/C++, or integrate Arduino CLI into editors like VS Code.
- This path does not orchestrate the Python/MPU side or Bricks — it is pure MCU development.

Workflow C: Standalone Linux (SBC mode) & CLI

Use the UNO Q like a Linux single-board computer:

- Attach an HDMI display, keyboard, and mouse via a PD-capable USB-C dongle, or connect over SSH once Wi-Fi is configured. VS Code + the Remote-SSH extension works well for browsing files on the board.

Drive the MPU-controlled LEDs directly from the Linux side, e.g. from the shell:

```
echo 1 | sudo tee /sys/class/leds/red:user/brightness # ON
echo 0 | sudo tee /sys/class/leds/red:user/brightness # OFF
```

-

Run Apps from the command line with the App Lab CLI:

```
arduino-app-cli app start ~/ArduinoApps/uno_q_blink # build + run both sides
arduino-app-cli app logs ~/ArduinoApps/uno_q_blink # view Python logs
arduino-app-cli app stop ~/ArduinoApps/uno_q_blink # stop (runs in background)
```

- The first run may take a while as Python/Arduino libraries install.

Additional References

- Hardware page: <https://docs.arduino.cc/hardware/uno-q/>
- UNO Q User Manual (Bridge/RPC, LEDs, SPI/I²C, standalone mode): <https://docs.arduino.cc/tutorials/uno-q/user-manual/>

- App Lab getting started:
<https://docs.arduino.cc/software/app-lab/tutorials/getting-started/>
 - Product overview & App Lab download: <https://www.arduino.cc/en/uno-q>
 - Qualcomm QRB2210 overview:
<https://www.qualcomm.com/products/internet-of-things/robotics-processors/qrb2210>
 - STM32U585 datasheet: <https://www.st.com/resource/en/datasheet/stm32u585ai.pdf>
 - Project Hub (examples): <https://projecthub.arduino.cc/?value=UNO%20Q>
 - Forum: <https://forum.arduino.cc/c/hardware/uno-familyd/uno-q/222>
-

Tip for teams: pick your track, start from the closest built-in Example, "Copy and edit," then pull in Bricks (Camera, Object Detection, Web Server) to add features fast. Keep heavy/AI/networking work in Python on the MPU and timing-critical hardware control in the sketch on the MCU, and let the Bridge move data between them.

AI Hub

Snapdragon Multiverse Hackathon: Qualcomm AI Hub Developer Guide

This guide covers the Qualcomm AI Hub workflow — how to find a model, get it compiled and quantized for a Snapdragon target, download the on-device bundle, and profile it — as one leg of your multi-device build for this hackathon (alongside the AI PC, mobile device, Arduino UNO Q, and Qualcomm Cloud AI 100).

“Source note: This guide uses only the public `qai-hub` / `qai-hub-models` Python packages and `onnxruntime` — no special client software is required. Every command here works from a plain `pip` install on your own laptop. For the canonical model catalog and account setup, always cross-check app.aihub.qualcomm.com.”

Table of Contents

- [Pre-work](#)
- [Core Architectural Concepts](#)
 - [What AI Hub Actually Does](#)
 - [The Build Host vs Run Host Split](#)
 - [Two Credentials You'll Need](#)
- [Track 1: Classical Models \(Vision/Audio\) via AI Hub](#)
- [Track 2: LLM / GenAI Bundles via AI Hub Genie](#)
- [Optimizing a Model for a Specific Device](#)
- [Profiling on Real Hardware](#)
- [Common Failure Modes](#)

- [Additional References](#)
-

Pre-work

1. Create an AI Hub account and get your API token at

app.aihub.qualcomm.com → Account → API Token.

Treat this token as a secret — never commit it, never paste it into a shared channel.

2. Install the AI Hub CLI/SDK:

```
pip install qai-hub qai-hub-models
qai-hub configure --api_token <your-token>
```

- 3.

4. Know which chipset you're targeting. AI Hub compiles per exact device —

```
qai-hub list-devices | grep -i "<your device>"
```

- 5.

For this hackathon that's most likely your AI PC (Snapdragon X Elite) or your phone (Snapdragon 8 Elite).

6. Decide classical model vs LLM early — the two paths use different AI Hub export flows and have very different resource requirements (see Tracks below).

7. If you're on Windows-ARM (Snapdragon X Elite) already — read [The Build Host vs Run Host Split](#) before you try to build anything. This is the single most common setup mistake.
-

Core Architectural Concepts

What AI Hub Actually Does

Qualcomm AI Hub is a cloud compilation service. You give it a model (ONNX / PyTorch / TFLite) and a target device string, and it:

1. Converts and quantizes the model for that device's exact NPU/DSP version
2. Runs it on real cloud-hosted Snapdragon hardware and reports back real latency
3. Returns a downloadable, ready-to-run artifact (QNN context binary, or for LLMs a full Genie bundle with tokenizer + config)

This is different from converting locally with the QAIRT SDK on your own machine (`qairt-converter` / `snpe-onnx-to-dlc`, no cloud round-trip) — AI Hub does the compile in the cloud, against real target hardware, which is why it needs your API token and a network connection.

The Build Host vs Run Host Split

This is the fact that trips up the most teams, so read it twice:

“The machine that exports an LLM bundle and the machine that runs it are usually different machines — even though they’re often the same Snapdragon laptop.”

Why: the AI Hub LLM exporter (qai-hub-models) depends on torch, and torch has no Windows-ARM64 wheel. So you cannot run the exporter directly on a Snapdragon X Elite's native Windows-ARM64 Python — even though the X Elite is exactly where the resulting bundle runs.

Host type	Can it BUILD (export) a bundle?	Can it RUN a bundle?
Windows-ARM64 (X Elite, native Python)	✗ no torch wheel	✓ yes — this is the target
WSL2 on the same X Elite (arm64 Linux)	✓ torch has arm64 Linux wheels	— (copy bundle out to Windows side)
x86_64 Windows / macOS / Linux dev box	✓	—

Practical flow: export the bundle in WSL (or any x86/macOS/Linux box), then copy the output folder onto the X Elite's Windows filesystem and run it there with the native ARM64 Python + onnxruntime-genai.

To set up WSL on your X Elite laptop (one-time, admin PowerShell):


```
wsl --install
```

Then inside WSL:

```
python3 -m venv ~/aihub-build && source ~/aihub-build/bin/activate  
pip install -U qai-hub qai-hub-models
```

This split only applies to LLM/Genie bundle exports. Classical vision/audio models compiled via `qai_hub_models.models.<slug>.export --target-runtime qnn` don't need `torch` locally in the same heavy way — see [Track 1](#).

Two Credentials You'll Need

Credential	Needed for	Where to get it
AI Hub API token	Every AI Hub compile/export — classical or LLM	app.aihub.qualcomm.com → Account → API Token
Hugging Face token	Only for gated LLM weights (Llama, Qwen)	huggingface.co — accept the model's license on its page first, then generate a token

Open models need no HF token at all. If your team wants to move fast on the LLM track without dealing with license acceptance, start with an open model (e.g. Falcon) to prove the pipeline end-to-end before touching gated weights.

Configure the AI Hub token once (on whichever machine will run exports):

```
export QAI_HUB_API_TOKEN=<your-token>
qai-hub configure --api_token $QAI_HUB_API_TOKEN
```

For gated LLM weights, also set:

```
export HF_TOKEN=<your-huggingface-token>
```

Track 1: Classical Models (Vision/Audio) via AI Hub

Best for: image classification, object detection, segmentation, audio classification — models that don't need the LLM decode-loop machinery.

1. Find a model

Browse the catalog: app.aihub.qualcomm.com →

Models. Filter by task (classification, detection, segmentation) and check that a precompiled artifact exists for your target device.

2. Export/compile for your device

```
python -m qai_hub_models.models.<slug>.export \
  --target-runtime qnn \
  --device "<exact device label from qai-hub list-devices>"
```

This runs the compile in the cloud against real hardware and downloads the result — you'll get real, hardware-measured latency numbers back, not an estimate. Note the output path it prints; that's your compiled artifact.

3. Bring it into your app

The downloaded artifact is a QNN context binary. Load it directly via `onnxruntime` with the QNN execution provider:

```
import os, onnxruntime as ort

# The QNN EP ships alongside onnxruntime-qnn (pip install onnxruntime-qnn)
sess = ort.InferenceSession(
    "model.onnx",
    providers=["QNNExecutionProvider"],
    provider_options=[{"backend_path": "QnnHtp.dll"}], # libQnnHtp.so on Linux/Android
)
outputs = sess.run(None, {"input": your_input_array})
```

If you'd rather convert a model you already have (ONNX/PyTorch) locally instead of a cloud round-trip, use the QAIRT SDK directly:

```
qairt-converter --input_network model.onnx --output_path model_int8.dlc \
--input_list calibration_images.txt --quantization_overrides int8
```

4. Profile it

See [Profiling on Real Hardware](#) below.

Track 2: LLM / GenAI Bundles via AI Hub Genie

Best for: on-device chat, summarization, function-calling, or any generative text use case running locally on the NPU (no cloud inference call at runtime).

1. Set up a build host

Per [The Build Host vs Run Host Split](#), do this

in WSL (if you're on the X Elite) or any x86/macOS/Linux box:

```
python3 -m venv ~/aihub-build && source ~/aihub-build/bin/activate
pip install -U qai-hub qai-hub-models
export QAI_HUB_API_TOKEN=<token>
qai-hub configure --api_token $QAI_HUB_API_TOKEN
```

2. Find your exact device string and export

```
qai-hub list-devices | grep -i "<your device>"

python -m qai_hub_models.models.<slug>.export \
  --target-runtime genie \
  --device "<exact device label>"
```

Use the model's package slug (underscores), not its friendly display name — e.g. `qai_hub_models.models.falcon_v3_7b_instruct.export` for "Falcon v3 7B Instruct". Note the output directory — it contains `genai_config.json`, the compiled graph/context binary, and the tokenizer.

Gated weights (Llama, Qwen): accept the license on the model's Hugging Face page and export HF_TOKEN before running the export. Open models (e.g. Falcon) need no HF token — the best first model to validate your pipeline end-to-end.

This export is large (multi-GB download + compile time) — budget time for it, ideally the night before rather than during your build window.

3. Copy the bundle to the run host

If you built in WSL, the Windows filesystem is under /mnt/c:

```
DST=/mnt/c/<your-app>/models/<friendly-model-id>
mkdir -p "$DST"
cp -r <export-output-dir>/ * "$DST"/
```

For any other build/run host split (e.g. built on a teammate's x86 laptop, running on your X Elite), just copy the folder over any way you like — USB drive, shared network folder, scp if both machines are reachable over SSH. The destination folder name should be your model's friendly id — whatever your runtime code looks up by.

4. Load and run it on the NPU

Using onnxruntime-genai (install with pip install onnxruntime-genai on the run host — native ARM64 Python on an X Elite, not an emulated x86 shell):

```

import onnxruntime_genai as og

model = og.Model("/path/to/bundle_dir") # the folder with genai_config.json
tokenizer = og.Tokenizer(model)

prompt = "What is the Snapdragon X Elite?"
tokens = tokenizer.encode(prompt)

params = og.GeneratorParams(model)
params.set_inputs(tokens)
params.set_search_options(max_length=200)

generator = og.Generator(model, params)
while not generator.is_done():
    generator.compute_logits()
    generator.generate_next_token()
    print(tokenizer.decode([generator.get_next_tokens()[0]]), end="", flush=True)

```

5. Verify it actually runs on the NPU

Don't assume from a clean load — check the real execution provider directly:

```

import os, onnxruntime_qnn as q, onnxruntime as o
os.add_dll_directory(os.path.dirname(q.__file__))
o.register_execution_provider_library("QNNExecutionProvider", q.get_library_path())
npu_devices = [d for d in o.get_ep_devices()
                if d.ep_name == "QNNExecutionProvider" and str(d.device.type).endswith("NPU")]
print("NPU device found:", bool(npu_devices))

```

If it falls back to CPU silently, your demo will look "done" but won't show the actual NPU speedup judges are scoring on Technical Implementation (40 points — explicitly includes "resource utilization, optimization, latency and performance, and energy efficiency"). Always verify the execution provider

directly rather than trusting that "it loaded" means "it's on the NPU." Note that in `onnxruntime-qnn 2.x`, the QNN EP does not show up in `onnxruntime.get_available_providers()` even when it's working correctly — you must check via `get_ep_devices()` as shown above.

Optimizing a Model for a Specific Device

Getting a model to *compile* for a device is the easy part — `--device "<label>"` handles that. Getting it to actually run well on that device's specific NPU/DSP generation is a separate set of choices. This is also exactly what the Technical Implementation judging criterion (40 points — "resource utilization, optimization, latency and performance, and energy efficiency") is scoring, so it's worth doing properly rather than just accepting AI Hub's defaults.

1. Match quantization to the device's NPU generation

Different Hexagon NPU versions have different native precision support. Compiling FP32 to a device whose NPU only accelerates INT8 means large parts of your graph silently fall back to CPU — slow, and higher power draw.

Device (this hackathon)	NPU	Native precision	Recommended quantization
Snapdragon X Elite (AI PC)	Hexagon v73, 45 TOPS	INT8 (INT4 for weights on LLMs)	INT8 for vision/audio; INT4 weight-only + FP16 activations for LLMs

Snapdragon 8 Elite (mobile)	Hexagon v73–v81 depending on exact chip	INT8	INT8
Arduino UNO Q (QCS2210)	Hexagon DSP v66, 1.0 TOPS	INT8 only	INT8 — mandatory; FP32 does not run on this DSP generation at all and silently falls back to CPU

Rule of thumb: always quantize to at least INT8 before compiling for any of this hackathon's NPU targets. INT8 typically gives $\geq 4\times$ size reduction with $< 1\%$ accuracy loss on classical vision models. Only go to INT4 for LLM weights (not classical vision models), and only if you can tolerate a larger accuracy drop.

2. Provide real calibration data — not random noise

Static quantization needs a calibration pass to compute per-tensor scale/zero-point values. If you calibrate with random data, your scales will be wrong and accuracy silently degrades even though the pipeline "works":

```
python -m qai_hub_models.models.<slug>.export \
  --target-runtime qnn \
  --device "<device label>" \
  --calibration-data ./calibration_images/ # real, representative inputs
```

Or with `onnxruntime.quantization.quantize_static` locally, pass a `CalibrationDataReader` backed by real sample inputs rather than random arrays.

Use ~10–100 real samples that match your actual inference-time input distribution (same resolution, same preprocessing, same domain — e.g. actual crowd photos if your model classifies crowd density, not generic ImageNet samples).

3. Check op coverage — don't assume 100% lands on the NPU

Most AI Hub export commands print a per-op breakdown of what compiled onto the target NPU vs what will fall back to CPU. Read this output — don't skip past it:

- $\geq 80\%$ coverage $\rightarrow$ NPU is still the right call; the few CPU-fallback ops won't dominate latency.
- $< 80\%$ coverage $\rightarrow$ the CPU $\leftrightarrow$ NPU round-trips for fallback ops may cost more than they save. Consider running on GPU instead (no CPU $\leftrightarrow$ NPU boundary crossing cost), or restructure the model to avoid the unsupported op (e.g. swap a custom activation for a standard one).

You can also inspect this directly with `onnxruntime`'s session after loading — `sess.get_providers()` shows which providers are actually attached, and verbose logging (`ort.set_default_logger_severity(0)`) will print the partition map showing which nodes were assigned to which execution provider.

4. Pick a power mode that matches your use case, not just "fastest"

Use case	Recommended profile	Why

Real-time camera/video pipeline	burst / performance	Pins NPU clock, avoids frame-to-frame latency variance from thermal throttling
Single interactive inference (e.g. chat turn, one photo)	balanced	Good latency without sustained high power draw
Battery-powered / sustained background sensing	power_saver / efficiency	Lowest power; latency variance is acceptable since nothing is real-time

(Exact flag names vary by tool — `snpe-net-run --perf_profile <mode>`, `qnn-net-run --perf_profile <mode>`; check `--help` on whichever CLI you're using.)

Running everything in `performance/burst` mode for a demo will look fast, but a judge asking about energy efficiency (explicitly scored) will catch a mismatch between your use case and your power profile choice — pick deliberately and be ready to explain why.

5. For LLMs specifically: profile per-token, not per-inference

A single "latency" number is meaningless for generative text. Report:

- Time-to-first-token (prefill latency) — matters for perceived responsiveness
- Tokens/sec during decode (steady-state generation speed) — matters for how fast the full response streams in

Both numbers should be captured from the actual device, not the AI Hub cloud compile-time estimate — see [Profiling on Real Hardware](#).

Profiling on Real Hardware

Once your model is compiled and deployed, get real numbers for your demo and submission writeup — don't rely on AI Hub's cloud compile-time estimate alone; capture numbers from the actual device you'll demo on.

Simple timing in Python (works anywhere `onnxruntime` runs):

```
import time
import numpy as np

# warmup
for _ in range(3):
    sess.run(None, {input_name: sample_input})

times = []
for _ in range(50):
    t0 = time.perf_counter()
    sess.run(None, {input_name: sample_input})
    times.append((time.perf_counter() - t0) * 1000)

times = np.array(times)
print(f"mean={times.mean():.1f}ms p50={np.percentile(times,50):.1f}ms "
      f"p95={np.percentile(times,95):.1f}ms p99={np.percentile(times,99):.1f}ms")
```

On the Arduino UNO Q board specifically, use the SNPE CLI's built-in benchmarking (see the [UNO Q guide](#) for the full on-device workflow):

```
snpe-net-run --container model_int8.dlc --input_list inputs.txt \
--use_dsp --duration 30 --debug
```

`--debug` prints per-layer timing; parse `Total Inference` for end-to-end latency.

For a visual profile (Chrome trace of kernel-level execution), QNN's `qnn-profile-viewer` tool (bundled with the QAIRT SDK) converts a profiling log into a trace you can drag into `chrome://tracing`.

Capture at minimum: mean/p50/p95/p99 latency, throughput (inferences/sec or tokens/sec), and — if you can measure it (many dev boards expose this via `/sys/class/power_supply/` or a USB power meter) — power draw. This is exactly what the Technical Implementation judging criterion is scoring, so having real numbers (not estimates) for your presentation matters.

Common Failure Modes

Symptom	Cause → Fix
No matching distribution found for <code>onnxruntime-genai-qnn</code>	That package name doesn't exist. The correct package is <code>onnxruntime-genai</code> — its <code>win-arm64</code> wheel already carries Hexagon support.
<code>pip</code> can't find a <code>win_arm64</code> wheel for <code>torch</code>	You're trying to build an LLM bundle directly on the X Elite. Build in WSL/x86/macOS/Linux

	instead — see Build Host vs Run Host.
quantized checkpoint ... must have an ONNX model	Wrong export flag. LLMs need <code>--target-runtime genie</code>, not the vision <code>--target-runtime qnn</code>.
403 / access denied during export	Gated weights. Accept the license on the model's Hugging Face page and set <code>HF_TOKEN</code>, or use an open model like Falcon.
Execution provider reports CPU after a "successful" load	The QNN EP library wasn't registered/attached correctly. Re-check the verification snippet in Track 2 step 5 — don't trust "it loaded" as "it's on the NPU."
<code>onnxruntime.get_available_providers()</code> doesn't list QNN even though the NPU works	Expected in <code>onnxruntime-qnn 2.x</code> — it's a plugin EP and won't show there. Check via <code>get_ep_devices()</code> instead (see Track 2 step 5).
Export/compile takes forever or times out	AI Hub queues jobs against real hardware — this is a shared resource during the hackathon. Kick off large exports early and in parallel with other work.

Additional References

- Official AI Hub portal (account, token, model catalog, device list):
app.aihub.qualcomm.com
- qai-hub / qai-hub-models PyPI packages and their own --help output are the most current reference for exact CLI flags — they change between SDK versions.
- Qualcomm SNPE / QNN SDK developer docs (for local, non-cloud conversion)
- Companion guide for the Arduino UNO Q leg of this hackathon:
[docs/UNOQ_HACKATHON_DEVELOPER_GUIDE.md](#)

Event Information Guide

Table of Contents

..... 1

Event Information Guide..... 1

Event Details..... 2

Overview 2

Partners 2

Event Description 2

Target Devices..... 3

Timeline 3

Event Schedule 3

Logistics..... 3

Event Kick-off / Master Class..... 4

Loaner Agreements..... 4

Mentors 4

Team Demos 4

Judging 4

Evaluation Criteria..... 5

Prizes 5

Project Submission Requirements..... 5

Swag..... 6

Resources..... 7

Qualcomm Developer Resources..... 7

DevRel AI Sample Apps 7

AI Hub 8

Third-Party Tools 8

Participant-provided Tools 8

Previous Hackathon Apps 8

Support..... 9

Event Details

Overview

Snapdragon Multiverse Hackathon

July 11 - 12, 2026

Location: Qualcomm Building N | Café

Address: Phase 2, Brookefield, Bengaluru, Karnataka 560066, India

Target Platforms: AI PC, mobile device, Arduino Uno Q, any other Snapdragon device or multicontroller + QC AI 100 Cloud

Hackathon Goal: Develop AI experiences across multiple form factors in the Qualcomm ecosystem

Partners

- Sarvam
- CodeMate
- OnePlus

Event Description

Step into the Snapdragon Multiverse—Qualcomm’s flagship hackathon and one of our largest developer events of the year—bringing together top builders, students, and innovators to design the next generation of multi-device AI experiences across compute, mobile, IoT, and cloud. Building on the success of previous Multiverse events, this is your opportunity to work hands-on with Snapdragon platforms and collaborate with a highly driven developer community to push the boundaries of connected computing.

Every team will have access to a Copilot+ PC powered by Snapdragon® X Series processors as the central control hub, a mobile device, Arduino UNO Q, and Qualcomm AI Cloud 100. Want to go further? Bring your own Snapdragon-powered devices or microcontrollers to craft seamless, intelligent cross-platform solutions.

Over two days, selected teams of 3–5 will build real-world applications that orchestrate intelligence across devices—from AI PCs and smartphones to microcontrollers and cloud infrastructure—solving meaningful problems grounded in the India ecosystem.

Unlike traditional hackathons, there are no predefined tracks—teams are free to explore any use case, as long as it results in a meaningful, real-world application. Whether you’re building productivity tools, intelligent assistants, smart infrastructure solutions, or entirely new multi-device experiences, the focus is on how systems work together, not just what runs on a single device.

Target Devices

- AI PC: Surface Laptop 7 13-inch XElite/32GB Ram /512GB SSD
- Mobile device: One Plus 15
- Arduino UNO Q

Timeline

Registration / Submission period: May 27 – June 28 (11:59 p.m.)

Shortlisting announcement: July 1

Pre-hackathon workshop / FAQ session: July 6

Application submission period: July 11 (1:00pm) – July 12 (1:00pm)

Application judging period: July 12 (1:01pm – 5 pm)

Winner announcement: July 12 (on or around 5 pm)

Event Schedule

Exact times listed below are subject to change.

Saturday, July 11

9am | QC arrival

10am | Check-in

11am | Event Kick-off & Introduction

11:15-11:30am | Masterclass by Qualcomm DevRel

11:30-12 pm | Sarvam – Edge and Hybrid Deployments

12-1 pm | Lunch and Device Distribution

1:00 pm | **Hack begins**

4-5 pm | Snacks

7:30-8:30pm | Dinner

12am | Late Night Snacks

Sunday, July 12

7-9am | Breakfast

12-1pm | Lunch

1:00 pm | **Application submission deadline**

1 – 4 pm | Team App Demos

4 – 4:15 pm | Device collection

4 :15– 4:30 pm | Judging & Finalization of Winners

4:30 – 5 pm | Felicitation Ceremony

5 – 7 pm | Social reception on campus

7pm onwards | Event Close & Wrap-up

Logistics

- Meals, coffee, and refreshments will be provided throughout the event.

- Teams may stay overnight in the Qualcomm venue (Saturday night only).
- Teams may take their Qualcomm-provided devices as long as the Team Lead has signed and returned the Loaner Agreement. All devices must be returned at the end of the event.
- Travel reimbursement is not provided for this event.

Event Kick-off / Master Class

Event kickoff will include introductions from the sponsors and a master class - an overview of Qualcomm's AI Stack, recommended tools to use during the hack, and a brief walk through of a sample app.

Loaner Agreements

Each team that takes one or more devices will need to complete a loaner agreement. One person from the team, the Team Lead, should fill out the form (name, home address, and signature) and return to a Qualcomm employee prior to receiving a device.

All devices must be returned by the end of the event. By signing the loaner agreement, you agree to return the device in good condition. If all devices are not returned, participant contact information will be shared with the Qualcomm Legal team for further action.

Mentors

5-10 industry experts will be onsite and available to help teams when they are stuck. Mentors are available to help teams stay unblocked and on track – not to build their project for them.

Team Demos

Each team will have 5 minutes to talk about and demo their application. They should highlight the technology used especially edge technologies. Each demo will be timed and be held to only 5 minutes, so each team is able to present.

Judging

We will have 3 winning teams. After each team has a chance to demo, each team will vote for their favorite application (one vote only and can't vote for their own team) and the judges will also evaluate each application. Highest scoring application will win the Top Award and the team with the most votes from participants will win the Team's Choice Award.

In addition, there will be an award to the team with the best use of multi-device orchestration. Each team is only eligible to win one prize.

We will have 5-7 judges for the event. Judges can also be mentors.

Evaluation Criteria

Submissions will be judged on the following criteria for the Top Prize:

- i. **Technical Implementation** (40 points)
Evaluation based on resource utilization, optimization, latency and performance, and energy efficiency.
- ii. **Application Use-Case and Innovation** (25 points)
Evaluation through the lens of problem solving, creativity and uniqueness, and user experience.
- iii. **Deployment and Accessibility** (20 points)
Evaluation based on ease of installation and use.
- iv. **Presentation and Documentation** (15 points)
Evaluation based on the clarity of explanation during the presentation, and code quality and documentation.

Submissions will be judged on the following criterion for the Multi-Device Prize category:

- i. **Multi-Device Orchestration Excellence** (100 points)
Evaluation based on cross-device coordination, seamless user experience, technical depth of orchestration, innovation and usefulness of the multi-device behavior, and reliability and polish.

Prizes

Top Prize – Overall winner (selected by judges):

- One (1) Copilot+ PC powered by Snapdragon X2 Elite for each member of the team
- Qualcomm DevRel support to complete application
- Blog + Live Stream opportunities

Multi-Device Prize Category (selected by judges for best use of multi-device orchestration):

- One (1) Ray-Ban Meta AI Glasses for each member of the team
- Qualcomm DevRel support to complete application
- Blog + Live Stream opportunities

Popularization Award (team popular vote):

- One (1) Arduino UNO Q for each member of the team
- Qualcomm DevRel support to complete application
- Blog + Live Stream opportunities

*Each team can only win one prize

Project Submission Requirements

To be considered for the prize, your submission must meet the following basic criteria:

- i. The application shall not include any closed-source existing code; all codes shall be open for consumption and available to the public.
- ii. The application must be provided in a GitHub repository, with the following files in addition to your code:
 - 1. A README file with the following information:
 - a. An application description;
 - b. Names and emails of all Eligible Individuals on the Team;
 - c. Setup instructions from scratch, including dependencies if applicable;
 - d. Run and usage instructions; and
 - 2. An open-source license (please refer to <https://choosealicense.com> for help choosing).
- iii. The application must be runnable using your provided instructions.
- iv. The application and most components must run on the edge (hybrid edge/cloud is acceptable, but the majority should run locally on device).
- v. The application must be capable of being successfully installed and run on the Platforms for which it is intended and must function as depicted in the text description.
- vi. The application must be developed and/or commercially ready to the extent that it may be deployed on app store or other open source platform for users to download.
- vii. The GitHub repository containing your application must be submitted by the Submission Period. Such GitHub repository must be submitted via Microsoft Forms. The link to the Form will be provided by Sponsor at the beginning of the onsite event.
- viii. (Optional) – The following components are highly recommended but not mandatory:
 - 3. Tests and testing instructions to verify the app setup;
 - 4. Notes section containing additional information not covered in the application description;
 - 5. References used while developing the application; and
 - 6. Well-commented code.

Swag

Swag will be available for all participants

Resources

Qualcomm Developer Resources

Qualcomm Developer Home	https://qualcomm.com/developer
Windows on Snapdragon Core Developer Docs	https://docs.qualcomm.com/bundle/publicresource/topics/80-62010-1/core-app-overview-.html?product=1601111740057789
Windows on Snapdragon AI Developer Docs	https://docs.qualcomm.com/bundle/publicresource/topics/80-62010-1/ai-app-development.html?product=1601111740057789
Qualcomm Developer Projects	Awesome Qualcomm Developer Projects
Qualcomm AI Inference Suite (Cloud)	https://www.qualcomm.com/developer/software/qualcomm-ai-inference-suite
Arduino UNO Q	https://www.qualcomm.com/developer/hardware/arduino-uno-q
Arduino UNO Q Project Hub	https://projecthub.arduino.cc/?value=UNO+Q

DevRel AI Sample Apps

Extensible python sample apps that can be forked as starting points for projects. Star to save for later!

Simple NPU Chatbot w/ AnythingLLM	https://github.com/thatrandomfrenchdude/simple_npu_chatbot
NPU Pose Detection w/ AI Hub	https://github.com/quic/Pose-Detection-with-HRPoseNet
Local Agent w/ LM Studio	https://github.com/thatrandomfrenchdude/local-agent
Simple Whisper Transcription w/ AI Hub	https://github.com/thatrandomfrenchdude/simple-whisper-transcription
Qualcomm AI Inference Suite (Cloud) Samples and Tutorials	https://docs.qualcomm.com/bundle/publicresource/topics/80-88545-1/index_tutorials.html?product=1601111740095226

AI Hub

Qualcomm AI Hub Models	https://aihub.qualcomm.com
AI Hub Getting Started	https://aihub.qualcomm.com/get-started
AI Hub Slack Community	https://qualcomm-ai-hub.slack.com/
AI Hub Model notebook	https://tinyurl.com/demo-aihub
AI Hub Bring Your Own Model notebook	https://tinyurl.com/byom-aihub

Third-Party Tools

Neo4J Graph Builder	https://github.com/neo4j-labs/llm-graph-builder/
AnythingLLM	https://anythingllm.com/
Microsoft AI Dev Gallery	aka.ms/ai-dev-gallery-store
LM Studio	https://lmstudio.ai

Participant-provided Tools

The following resources have been generated by teams participating in this or previous hacks and are made available to all participants to ensure fairness.

Voice Stress Detection Model	https://github.com/chhavi876/ShieldHer/tree/main/ai_model
------------------------------	---

Previous Hackathon Apps

Tutor.AI Sample App	https://github.com/nirmal141/tutor-ai
---------------------	---

R.E.D.A.C.T. Sample App	https://github.com/abhishekk962/redact
-------------------------	---

Support

We will have mentors available to ask questions in person.

Additionally, feel free to reach out on Qualcomm Discord for 24/7 support:

<https://discord.com/invite/qualcommdevelopernetwork> (#snapdragon-multiverse-hack-bangalore)